



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

LAB MANUAL
DEEP LEARNING LAB [CSE 3281]

Sixth Semester BTech in CSE(AI&ML)
(JAN – MAY 2024)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL-576104



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

CERTIFICATE

This is to certify that Ms./Mr.

Reg. No.: Section: Roll No.:

has satisfactorily completed the **LAB EXERCISES PRESCRIBED FOR DEEP LEARNING LAB (CSE 3281)** of Third Year B.Tech. degree in Computer Science and Engineering (AI & ML) at MIT, Manipal, in the Academic Year 2023– 2024.

Date:

Signature
Faculty in Charge

CONTENTS

LAB NO.	TITLE	PAGE NO.	REMARKS
	Course Objectives and Outcomes	i	
	Evaluation plan	i	
	Instructions to the Students	ii	
1	Introduction to tensors		
2	Computational graphs		
3	Deep Learning Library in PyTorch		
4	Feed-forward Neural Networks		
5	Convolutional Neural Network		
6	Transfer Learning		
7	Lab Mid Term + Mini-project Evaluation		
8	Regularization for Deep Neural Networks		
9	Recurrent Neural Networks		
10	Long-Short Term Memory (LSTM)		
11	Lab End Sem Exam		
12	Encoder-Decoders, Variational Auto Encoders		
13	Mini-Project – Final Evaluation		
	References		

Course Objectives

- Understand implementation detail of deep learning models.
- Develop familiarity with tools and software frameworks for designing DNNs.

Course Outcomes

At the end of this course, students will be able to

1. Demonstrate the use of PyTorch library to implement deep feed-forward networks and Convolutional Neural Network with activation functions for regression and classification problems.
2. Use pretrained models to design deep neural networks by creating/restoring checkpoints
3. Apply regularization techniques and parameter optimization to improve model performance.
4. Implement well-known deep neural architectures for NLP/Computer Vision applications using RNN and LSTM.
5. Illustrate the concepts of Auto-Encoders (AEs), Variational Auto-Encoders (VAEs), and Generative Adversarial Networks (GANs) for effective representation and data synthesis

Evaluation plan

- Internal Assessment Marks: 60M
 - Continuous Evaluation: 20M
 - Continuous evaluation component (for each evaluation): 10 marks
 - The assessment will depend on punctuality, program execution, maintaining the observation note and answering the questions in viva voce.
 - Mid-term test : 20M
 - Mini-project: 20M [Report 50% + Implementation and Demo 50%]
- End semester assessment: 40

INSTRUCTIONS TO THE STUDENTS

Pre- Lab Session Instructions

1. Students should carry the Lab Manual Book and the required stationery to every lab session.
2. Be in time and follow the institution dress code.
3. Must Sign in the log register provided.
4. Make sure to occupy the allotted seat and answer the attendance
5. Adhere to the rules and maintain the decorum.
6. Students must come prepared for the lab in advance.

In- Lab Session Instructions

- Follow the instructions on the allotted exercises.
- Show the program and results to the instructors on completion of experiments.
- On receiving approval from the instructor, copy the program and results in the Lab record.
- Prescribed textbooks and class notes can be kept ready for reference if required.

General Instructions for the exercise in Lab

- Implement the given exercise individually and not in a group.
- Observation book should be complete with program, proper input output clearly showing the parallel execution in each process. Plagiarism (copying from others) is strictly prohibited and would invite severe penalty in evaluation.
- The exercises for each week are divided under three sets:
 - Solved example
 - Lab exercises - to be completed during lab hours
 - Additional Exercises - to be completed outside the lab or in the lab to enhance the
- In case a student misses a lab class, he/ she must ensure that the experiment is completed during the repetition class with the permission of the faculty concerned but credit will be given only to one day's experiment(s).
- Questions for lab tests and examination are not necessarily limited to the questions in the manual, but may involve some variations and / or combinations of the questions.

THE STUDENTS SHOULD NOT

- Bring mobile phones or any other electronic gadgets to the lab.
- Go out of the lab without permission.

Lab No 1:

Date:

Introduction to tensors

Objectives:

In this lab, student will be able to

1. Setup pytorch environment for deep learning
2. Understand the concept of tensor
3. Manipulate tensors using built-in functions

A summary of the topics that is covered in this session are:

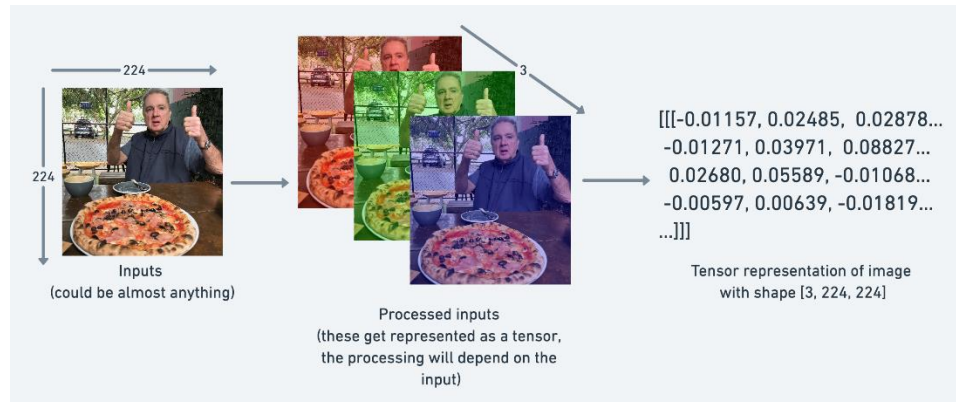
Topic	Contents
Introduction to tensors	Tensors are the basic building block of all of machine learning and deep learning.
Creating tensors	Tensors can represent almost any kind of data (images, words, tables of numbers).
Getting information from tensors	If you can put information into a tensor, you'll want to get it out too.
Manipulating tensors	Machine learning algorithms (like neural networks) involve manipulating tensors in many different ways such as adding, multiplying, combining.
Dealing with tensor shapes	One of the most common issues in machine learning is dealing with shape mismatches (trying to mixed wrong shaped tensors with other tensors).
Indexing on tensors	If you've indexed on a Python list or NumPy array, it's very similar with tensors, except they can have far more dimensions.
Mixing PyTorch tensors and NumPy	PyTorch plays with tensors (torch.Tensor), NumPy likes arrays (np.ndarray) sometimes you'll want to mix and match these.
Running tensors on GPU	GPUs (Graphics Processing Units) make your code faster, PyTorch makes it easy to run your code on GPUs.

Sample Exercise:

Use console window to execute the instructions given below:

```
import torch
torch.__version__
```

Introduction to tensors



Creating tensors

```
# Scalar
scalar = torch.tensor(7)
scalar
```

```
# Get the Python number within a tensor (only works with one-element tensors)
scalar.item()
```

```
# Vector
vector = torch.tensor([7, 7])
vector
```

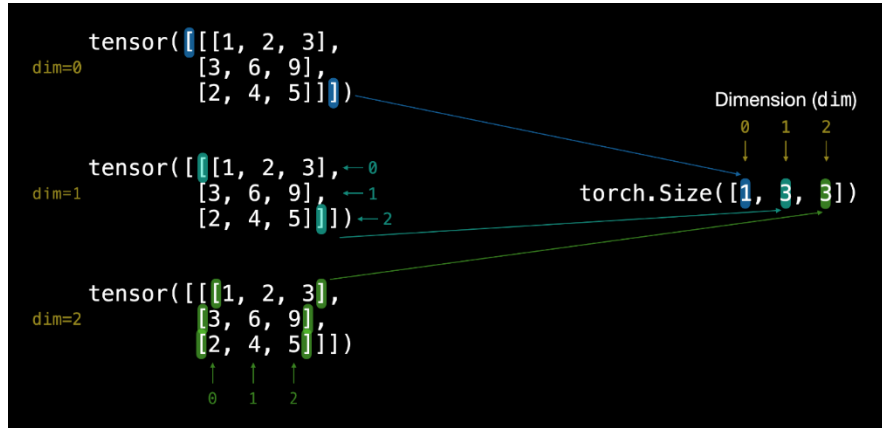
```
# Matrix
MATRIX = torch.tensor([[7, 8],  
                        [9, 10]])
MATRIX
```

```
MATRIX.shape
```

```
# Tensor
TENSOR = torch.tensor([[[1, 2, 3],  
                        [3, 6, 9],  
                        [2, 4, 5]]])
TENSOR
```

```
# Check number of dimensions for TENSOR
TENSOR.ndim
```

Visualization of Tensor Dimension:



Scalar

7

Vector

$\begin{bmatrix} 7 \\ 4 \end{bmatrix}$ or $\begin{bmatrix} 7 & 4 \end{bmatrix}$

Matrix

$\begin{bmatrix} 7 & 10 \\ 4 & 3 \\ 5 & 1 \end{bmatrix}$

Tensor

$\begin{bmatrix} \begin{bmatrix} 7 & 4 & 0 & 1 \end{bmatrix} \\ \begin{bmatrix} 1 & 9 & 2 & 3 \end{bmatrix} \\ \begin{bmatrix} 5 & 6 & 8 & 8 \end{bmatrix} \end{bmatrix}$

Random Tensors:

```
# Create a random tensor of size (3, 4)
random_tensor = torch.rand(size=(3, 4))
```

```
random_tensor, random_tensor.dtype
```

Output:

```
(tensor([[0.9900, 0.1882, 0.1744, 0.7445],
        [0.9445, 0.7044, 0.7024, 0.7877],
        [0.0218, 0.7861, 0.9037, 0.9690]]),
torch.float32)
```


The flexibility of `torch.rand()` is that we can adjust the size to be whatever we want.

For example, say you wanted a random tensor in the common image shape of `[224, 224, 3]` ([height, width, color_channels]).

```
# Create a random tensor of size (224, 224, 3)
random_image_size_tensor = torch.rand(size=(224, 224, 3))

random_image_size_tensor.shape, random_image_size_tensor.ndim
(torch.Size([224, 224, 3]), 3)
```

Zeros and ones

Sometimes you'll just want to fill tensors with zeros or ones.

This happens a lot with masking (like masking some of the values in one tensor with zeros to let a model know not to learn them).

Let's create a tensor full of zeros with `torch.zeros()`

Again, the size parameter comes into play.

```
# Create a tensor of all zeros
zeros = torch.zeros(size=(3, 4))
zeros, zeros.dtype
```

Output:

```
(tensor([[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]]),
torch.float32)
```

We can do the same to create a tensor of all ones except using `torch.ones()` instead.

```
# Create a tensor of all ones
ones = torch.ones(size=(3, 4))
ones, ones.dtype
```

Output:

```
(tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]]),
torch.float32)
```

Creating a range and tensors:

Sometimes you might want a range of numbers, such as 1 to 10 or 0 to 100. You can use `torch.arange(start, end, step)` to do so.

Where:

start = start of range (e.g. 0)

end = end of range (e.g. 10)

step = how many steps in between each value (e.g. 1)

Note: In Python, you can use `range()` to create a range. However in PyTorch, `torch.range()` is deprecated and may show an error in the future.

```
# Use torch.arange(), torch.range() is deprecated
zero_to_ten_deprecated = torch.range(0, 10) # Note: this may return an error in the future
```

```
# Create a range of values 0 to 10
zero_to_ten = torch.arange(start=0, end=10, step=1)
zero_to_ten
```

```
# Can also create a tensor of zeros similar to another tensor
ten_zeros = torch.zeros_like(input=zero_to_ten) # will have same shape
ten_zeros
```

Note:

There are many different tensor datatypes available in PyTorch. Some are specific for CPU and some are better for GPU. Getting to know which is which can take some time. Generally if you see `torch.cuda` anywhere, the tensor is being used for GPU (since Nvidia GPUs use a computing toolkit called CUDA). The most common type (and generally the default) is `torch.float32` or `torch.float`. This is referred to as "32-bit floating point". But there's also 16-bit floating point (`torch.float16` or `torch.half`) and 64-bit floating point (`torch.float64` or `torch.double`). And to confuse things even more there's also 8-bit, 16-bit, 32-bit and 64-bit integers. The reason for all of these is to do with precision in computing. Precision is the amount of detail used to describe a number. The higher the precision value (8, 16, 32), the more detail and hence data used to express a number. This matters in deep learning and numerical computing because you're making so many operations, the more detail you have to calculate on, the more compute you have to use. So lower precision datatypes

are generally faster to compute on but sacrifice some performance on evaluation metrics like accuracy (faster to compute but less accurate).

Let's see how to create some tensors with specific datatypes. We can do so using the dtype parameter.

```
# Default datatype for tensors is float32

float_32_tensor = torch.tensor([3.0, 6.0, 9.0], dtype=None, device=None,
requires_grad=False)

# dtype=None, defaults to None, which is torch.float32 or whatever datatype is
passed
# device=None, defaults to None, which uses the default tensor type
# requires_grad=False if True, operations performed on the tensor are recorded

float_32_tensor.shape, float_32_tensor.dtype, float_32_tensor.device

float_16_tensor = torch.tensor([3.0, 6.0, 9.0],
                                dtype=torch.float16) # torch.half would also
work

float_16_tensor.dtype

# Create a tensor
some_tensor = torch.rand(3, 4)

# Find out details about it
print(some_tensor)
print(f"Shape of tensor: {some_tensor.shape}")
print(f"Datatype of tensor: {some_tensor.dtype}")
print(f"Device tensor is stored on: {some_tensor.device}") # will default to
CPU

tensor([[0.9270, 0.6217, 0.9093, 0.1493],
        [0.4354, 0.6207, 0.9224, 0.0312],
        [0.3300, 0.0959, 0.6050, 0.7674]])
Shape of tensor: torch.Size([3, 4])
Datatype of tensor: torch.float32
Device tensor is stored on: cpu
```

Manipulating tensors (tensor operations)

In deep learning, data (images, text, video, audio, protein structures, etc) gets represented as tensors.

A model learns by investigating those tensors and performing a series of operations on tensors to create a representation of the patterns in the input data.

These operations are often:

- Addition
- Subtraction
- Multiplication (element-wise)
- Division
- Matrix multiplication

Basic operations

Let's start with a few of the fundamental operations, addition (+), subtraction (-), multiplication (*).

They work just as you think they would.

```
# Create a tensor of values and add a number to it
tensor = torch.tensor([1, 2, 3])
tensor + 10
tensor([11, 12, 13])

# Multiply it by 10
tensor * 10
tensor([10, 20, 30])
```

Notice how the tensor values above didn't end up being `tensor([110, 120, 130])`, this is because the values inside the tensor don't change unless they're reassigned.

```
# Tensors don't change unless reassigned
tensor
tensor([1, 2, 3])
Let's subtract a number and this time we'll reassign the tensor variable.
# Subtract and reassign
tensor = tensor - 10
tensor
tensor([-9, -8, -7])
# Add and reassign
tensor = tensor + 10
tensor
tensor([1, 2, 3])
```

PyTorch also has a bunch of built-in functions like `torch.mul()` (short for multiplication) and `torch.add()` to perform basic operations.

```
# Can also use torch functions
torch.multiply(tensor, 10)
tensor([10, 20, 30])
# Original tensor is still unchanged
tensor
tensor([1, 2, 3])
```

However, it's more common to use the operator symbols like `*` instead of `torch.mul()`

```
# Element-wise multiplication (each element multiplies its equivalent, index 0->0, 1->1, 2->2)
print(tensor, "*", tensor)
print("Equals:", tensor * tensor)
tensor([1, 2, 3]) * tensor([1, 2, 3])
Equals: tensor([1, 4, 9])
```

Matrix multiplication:

One of the most common operations in machine learning and deep learning algorithms (like neural networks) is matrix multiplication. PyTorch implements matrix multiplication functionality in the `torch.matmul()` method.

The main two rules for matrix multiplication to remember are:

1. The **inner dimensions** must match:
 - (3, 2) @ (3, 2) won't work
 - (2, 3) @ (3, 2) will work
 - (3, 2) @ (2, 3) will work
2. The resulting matrix has the shape of the **outer dimensions**:
 - (2, 3) @ (3, 2) -> (2, 2)
 - (3, 2) @ (2, 3) -> (3, 3)

Let's create a tensor and perform element-wise multiplication and matrix multiplication on it.

```
import torch
tensor = torch.tensor([1, 2, 3])
tensor.shape
torch.Size([3])
```

The difference between element-wise multiplication and matrix multiplication is the addition of values.

For our tensor variable with values [1, 2, 3]:

Operation	Calculation	Code
Element-wise multiplication	$[1*1, 2*2, 3*3] = [1, 4, 9]$	<code>tensor * tensor</code>
Matrix multiplication	$[1*1 + 2*2 + 3*3] = [14]$	<code>tensor.matmul(tensor)</code>

```
# Element-wise matrix multiplication
tensor * tensor
tensor([1, 4, 9])
# Matrix multiplication
torch.matmul(tensor, tensor)
tensor(14)
# Can also use the "@" symbol for matrix multiplication, though not recommended
tensor @ tensor
tensor(14)
```

You can do matrix multiplication by hand but it's not recommended.

The in-built `torch.matmul()` method is faster.

```
%%time
# Matrix multiplication by hand
# (avoid doing operations with for loops at all cost, they are computationally
expensive)
value = 0
for i in range(len(tensor)):
    value += tensor[i] * tensor[i]
value
CPU times: user 178 µs, sys: 62 µs, total: 240 µs
Wall time: 248 µs
```

```
tensor(14)
%%time
torch.matmul(tensor, tensor)
CPU times: user 272 µs, sys: 94 µs, total: 366 µs
Wall time: 295 µs
```

```
tensor(14)
```

Getting PyTorch to run on the GPU

You can test if PyTorch has access to a GPU using `torch.cuda.is_available()`.

```
# Check for GPU
import torch
torch.cuda.is_available()
False
```

Let's create a device variable to store what kind of device is available.

```
# Set device type
device = "cuda" if torch.cuda.is_available() else "cpu"
device
'cpu'
```

```
# Count number of devices
torch.cuda.device_count()
```

Putting tensors (and models) on the GPU

You can put tensors (and models, we'll see this later) on a specific device by calling `to(device)` on them. Where device is the target device you'd like the tensor (or model) to go to.

Why do this?

GPUs offer far faster numerical computing than CPUs do and if a GPU isn't available, because of our device agnostic code (see above), it'll run on the CPU.

Note: Putting a tensor on GPU using `to(device)` (e.g. `some_tensor.to(device)`) returns a copy of that tensor, e.g. the same tensor will be on CPU and GPU. To overwrite tensors, reassign them:

```
some_tensor = some_tensor.to(device)
```

Let's try creating a tensor and putting it on the GPU (if it's available).

```
# Create tensor (default on CPU)
tensor = torch.tensor([1, 2, 3])

# Tensor not on GPU
print(tensor, tensor.device)

# Move tensor to GPU (if available)
tensor_on_gpu = tensor.to(device)
tensor_on_gpu
```

```
tensor([1, 2, 3]) cpu
tensor([1, 2, 3], device='cuda:0')
```

Moving tensors back to the CPU

What if we wanted to move the tensor back to CPU?

For example, you'll want to do this if you want to interact with your tensors with NumPy (NumPy does not leverage the GPU).

Let's try using the `torch.Tensor.numpy()` method on our `tensor_on_gpu`.

```
# If tensor is on GPU, can't transform it to NumPy (this will error)
tensor_on_gpu.numpy()
```

Instead, to get a tensor back to CPU and usable with NumPy we can use `Tensor.cpu()`. This copies the tensor to CPU memory so it's usable with CPUs.

```
# Instead, copy the tensor back to cpu
tensor_back_on_cpu = tensor_on_gpu.cpu().numpy()
tensor_back_on_cpu
```

Lab Exercise:

1. Illustrate the functions for Reshaping, viewing, stacking, squeezing and unsqueezing of tensors
2. Illustrate the use of `torch.permute()`.
3. Illustrate indexing in tensors
4. Show how numpy arrays are converted to tensors and back again to numpy arrays
5. Create a random tensor with shape (7, 7).
6. Perform a matrix multiplication on the tensor from 2 with another random tensor with shape (1, 7) (hint: you may have to transpose the second tensor).
7. Create two random tensors of shape (2, 3) and send them both to the GPU (you'll need access to a GPU for this).
8. Perform a matrix multiplication on the tensors you created in 6 (again, you may have to adjust the shapes of one of the tensors).
9. Find the maximum and minimum values of the output of 7.
10. Find the maximum and minimum index values of the output of 7.

11. Make a random tensor with shape (1, 1, 1, 10) and then create a new tensor with all the 1 dimensions removed to be left with a tensor of shape (10). Set the seed to 7 when you create it and print out the first tensor and it's shape as well as the second tensor and it's shape.

Computation Graphs

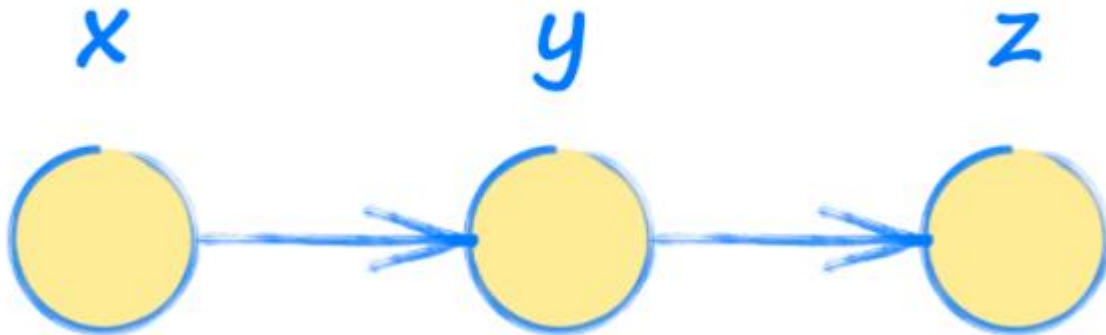
Objectives:

In this lab, student will be able to

- Calculate derivatives in PyTorch and perform auto differentiation on tensors.
- Calculate partial derivatives in PyTorch and implement the derivative of functions with respect to multiple values.
- Create a computation graph that involves different nodes and leaves to calculate the gradients using the chain rule to visualize the prediction (forward path) and parameter learning (backward path) in a step-wise fashion.
- To use computation graphs to make complicated mathematical concepts of learning algorithms more intuitive.

A summary of the topics that is covered in this session are:

To better understand neural networks, it is important to practice with computation graphs. These graphs are essentially a simplified version of neural networks with a sequence of operations used to see how the output of a system is affected by the input.



In other words, input x is used to find y , which is then used to find the output z .

PyTorch allows to automatically obtain the gradients of a tensor with respect to a defined function. When creating the tensor, we have to indicate that it requires the gradient computation using the flag `requires_grad`

Sample Program:

```
x = torch.rand(3,requires_grad=True)
print(x)
tensor([0.9207, 0.2854, 0.1424], requires_grad=True)
```

Notice that now the Tensor shows the flag `requires_grad` as `True`. We can also activate such a flag in a Tensor already created as follows:

```
x = torch.tensor([1.0,2.0,3.0])
x.requires_grad_(True)
print(x)
tensor([1., 2., 3.], requires_grad=True)
```

Problem 1:

Consider that y and z are calculated as follows:

$$y = x^2$$

$$z = 2y + 3$$

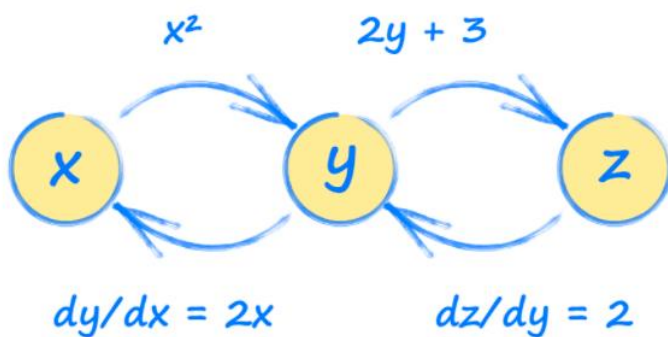
We are interested in how output z changes with input x

$$\frac{dz}{dx} = \frac{dz}{dy} * \frac{dy}{dx}$$

$$\frac{dz}{dx} = 2.2x$$

$$\frac{dz}{dx} = 4x$$

For input $x=3.5$, will make $z = 14$



This picture is called a computation graph. Using this graph, we can see how each tensor will be affected by a change in any other tensor. These relationships are gradients and are used to update a neural network during training

```
import torch

# set up simple graph relating x, y and z
x = torch.tensor(3.5, requires_grad=True)
y = x*x
z = 2*y + 3

print("x: ", x)
print("y = x*x: ", y)
print("z= 2*y + 3: ", z)

# work out gradients
z.backward()

print("Working out gradients dz/dx")

# what is gradient at x = 3.5
print("Gradient at x = 3.5: ", x.grad)
```

```
x: tensor(3.5000, requires_grad=True)
y = x*x: tensor(12.2500, grad_fn=<MulBackward0>)
z = 2*y + 3: tensor(27.5000, grad_fn=<AddBackward0>)
```

Working out gradients dz/dx

Gradient at x = 3.5: tensor(14.)

Problem 2:

Consider the function $f(x)=(x-2)^2$. Compute $d/dx f(x)$ and then compute $f'(1)$. Write code to check analytical gradient.

```
def f(x):
    return (x-2)**2

def fp(x):
    return 2*(x-2)
```

```
x = torch.tensor([1.0], requires_grad=True)
y = f(x)
y.backward()
```

```
print('Analytical f'(x):', fp(x))
print('PyTorch's f'(x):', x.grad)
```

Analytical $f'(x)$: tensor([-2.], grad_fn=<MulBackward0>)

PyTorch's $f'(x)$: tensor([-2.])

Problem 3:

Define a function $y = x^2 + 5$. The function `y` will not only carry the result of evaluating x , but also the gradient function $\frac{\partial y}{\partial x}$ called `grad_fn` in the new tensor `y`. Compare the result with analytical gradient.

```
x = torch.tensor([2.0])
x.requires_grad_(True) #indicate we will need the gradients with respect to this variable
y = x**2 + 5
print(y)
```

tensor([9.], grad_fn=<AddBackward0>)

To evaluate the partial derivative $\frac{\partial y}{\partial x}$, we use the `.backward()` function and the result of the gradient evaluation is stored in `x.grad`

```
y.backward() #dy/dx
print('PyTorch gradient:', x.grad)
```

#Let us compare with the analytical gradient of $y = x^2 + 5$

with torch.no_grad(): #this is to only use the tensor value without its gradient information

dy_dx = 2*x #analytical gradient

print('Analytical gradient:',dy_dx)

PyTorch gradient: tensor([4.])

Analytical gradient: tensor([4.])

Problem 4:

Write a function to compute the gradient of the sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$

Write $\sigma(x)$ as a composition of several elementary functions, as $\sigma(x) = s\left(c\left(b\left(a(x)\right)\right)\right)$

where: $a(x) = -x$

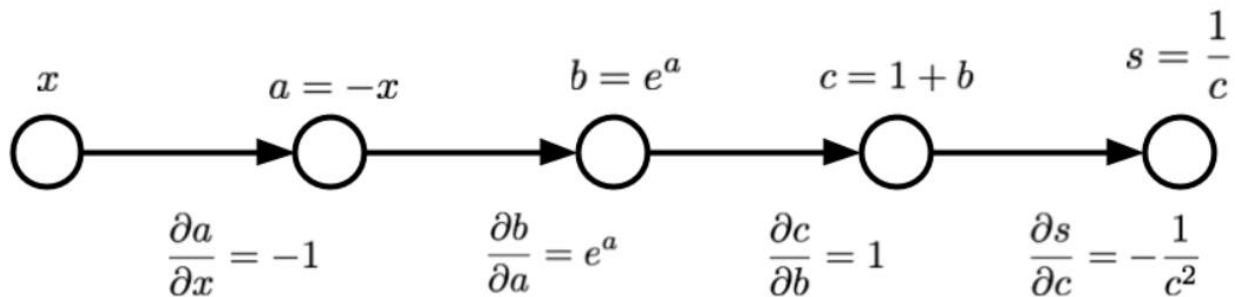
$b(a) = e^a$

$c(b) = 1 + b$

$s(c) = \frac{1}{c}$

It contains several intermediate variables, each of which are basic expressions for which we can easily compute the local gradients.

The computation graph for this expression is shown in the figure below



The input to this function is x , and the output is represented by node s . Compute the gradient of s with respect to x , $\frac{\partial s}{\partial x}$. In order to make use of our intermediate computations, we can use the chain rule as follows:

$$\frac{\partial s}{\partial x} = \frac{\partial s}{\partial c} \frac{\partial c}{\partial b} \frac{\partial b}{\partial a} \frac{\partial a}{\partial x}$$

```
def grad_sigmoid_manual(x):
    """Implements the gradient of the logistic sigmoid function
    #sigma(x) = 1 / (1 + e^{-x})
    """
    # Forward pass, keeping track of intermediate values for use in the
    # backward pass
    a = -x      # -x in denominator
    b = np.exp(a) # e^{-x} in denominator
    c = 1 + b    # 1 + e^{-x} in denominator
    s = 1.0 / c  # Final result, 1.0 / (1 + e^{-x})

    # Backward pass
    dsdc = (-1.0 / (c**2))
    dsdb = dsdc * 1
    dsda = dsdb * np.exp(a)
    dsdx = dsda * (-1)

    return dsdx

def sigmoid(x):
    y = 1.0 / (1.0 + torch.exp(-x))
    return y

input_x = 2.0
```

```
x = torch.tensor(input_x).requires_grad_(True)
y = sigmoid(x)
y.backward()
```

Compare the results of manual and automatic gradient functions:

```
print('autograd:', x.grad.item())
print('manual:', grad_sigmoid_manual(input_x))
```

autograd: 0.10499356687068939

manual: 0.1049935854035065

Exercise Questions:

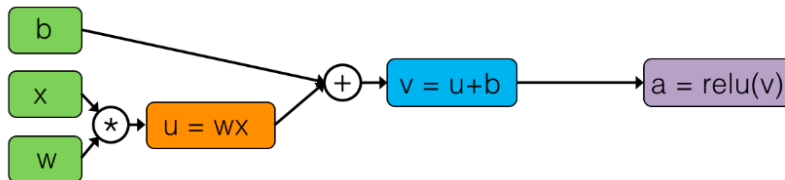
1. Draw Computation Graph and work out the gradient dz/da by following the path back from z to a and compare the result with the analytical gradient.

$$x = 2*a + 3*b$$

$$y = 5*a*a + 3*b*b*b$$

$$z = 2*x + 3*y$$

2. For the following Computation Graph, work out the gradient da/dw by following the path back from a to w and compare the result with the analytical gradient.



3. Repeat the Problem 2 using Sigmoid function
4. Verify that the gradients provided by PyTorch match with the analytical gradients of the function $f = \exp(-x^2 - 2x - \sin(x))$ w.r.t x
5. Compute gradient for the function $y = 8x^4 + 3x^3 + 7x^2 + 6x + 3$ and verify the gradients provided by PyTorch with the analytical gradients. A snapshot of the Python code is provided below.

$$8x^4 + 3x^3 + 7x^2 + 6x + 3$$

$$\begin{aligned} & \left| \frac{d}{dx} [8x^4 + 3x^3 + 7x^2 + 6x + 1] \right. \\ &= 8 \cdot \frac{d}{dx} [x^4] + 3 \cdot \frac{d}{dx} [x^3] + 7 \cdot \frac{d}{dx} [x^2] + 6 \cdot \frac{d}{dx} [x] + \frac{d}{dx} [1] \\ &= 8 \cdot 4x^3 + 3 \cdot 3x^2 + 7 \cdot 2x + 6 \cdot 1 + 0 \\ &= 32x^3 + 9x^2 + 14x + 6 \end{aligned}$$

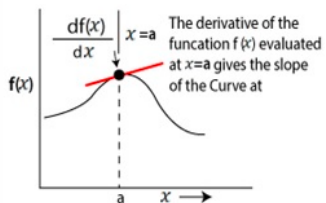
Finding derivative

$$32 \cdot (2)^3 + 9 \cdot (2)^2 + 14 \cdot 2 + 6$$

$$256 + 36 + 28 + 6$$

$$326$$

Derivative
 $\frac{df(x)}{dx}$



```
import torch
```

```
x=torch.tensor(2.0, requires_grad=True)
```

```
y=8*x**4+3*x**3+7*x**2+6*x+3
```

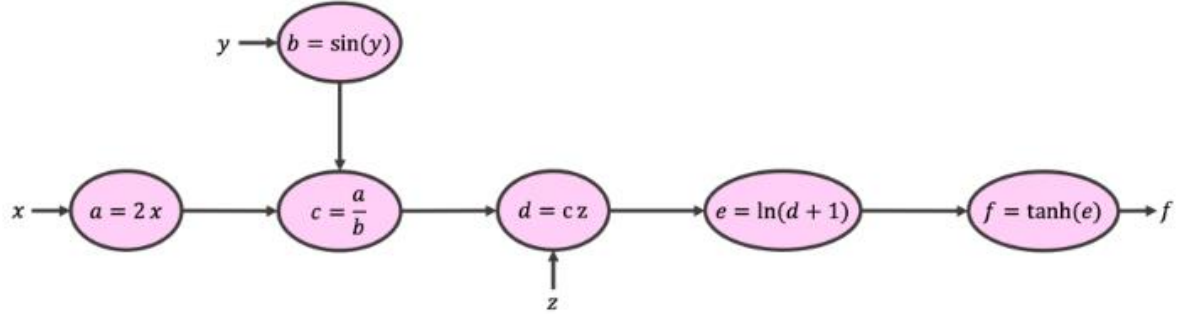
```
y.backward()
```

```
x.grad
```

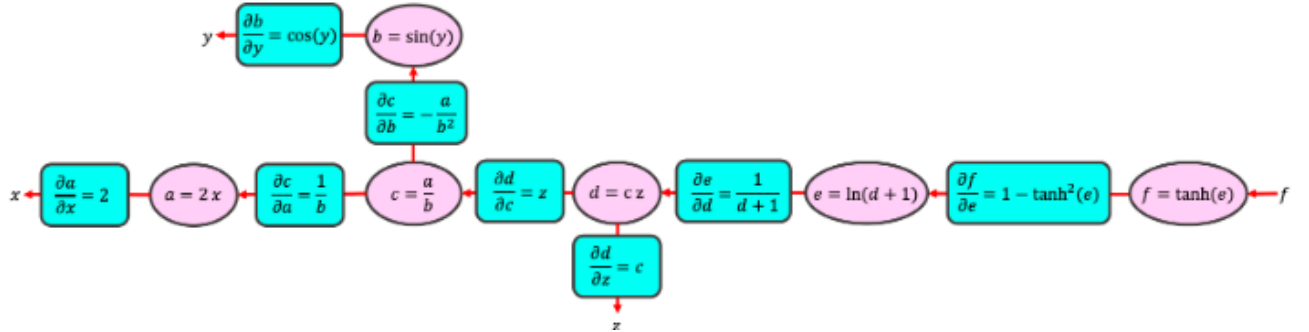
```
tensor(326.)
```


6. For the following function, computation graph is provided below.

$$f(x, y, z) = \tanh\left(\ln\left[1 + z \frac{2x}{\sin(y)}\right]\right)$$



Calculate the intermediate variables a, b, c, d, and e in the forward pass. Starting from f, calculate the gradient of each expression in the backward pass manually. Calculate $\partial f / \partial y$ using the computational graph and chain rule. Use the chain rule to calculate gradient and compare with analytical gradient.



$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial c} \frac{\partial c}{\partial a} \frac{\partial a}{\partial x} = (1 - \tanh^2(e)) \cdot \frac{1}{d+1} \cdot z \cdot \frac{1}{b} \cdot 2$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial c} \frac{\partial c}{\partial b} \frac{\partial b}{\partial y} = (1 - \tanh^2(e)) \cdot \frac{1}{d+1} \cdot z \cdot \frac{-a}{b^2} \cdot \cos(y)$$

Deep Learning Library in PyTorch

Objectives:

In this lab, student will be able to

- To build neural networks from scratch, starting off with a simple linear regression model.
- Develop PyTorch deep learning models for predictive modeling tasks such as linear regression and classification.
- Using the PyTorch API for deep learning model development tasks such as linear regression
- To explore multiple ways of implementing linear regression and logistic regression using PyTorch

Linear regression

Linear regression is a linear model, a model that assumes a linear relationship between the input variables (x) and the single output variable (y). More specifically, that y can be calculated from a linear combination of the input variables (x).

- When there is a single input variable (x), the method is referred to as simple linear regression.
- When there are multiple input variables, multiple linear regression.

Allows us to understand relationship between two continuous variables

Example

x: independent variable

weight

y: dependent variable

height

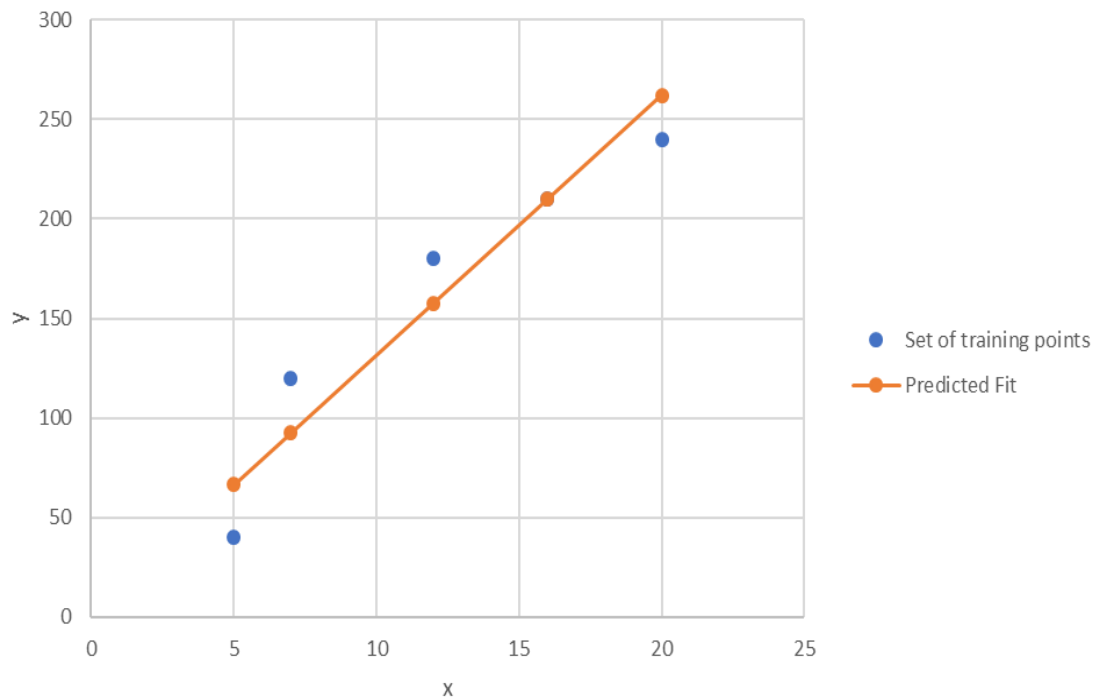
$y=wx+b$

Aim of Linear Regression: Minimize the distance between the points and the line ($y=\alpha x+\beta$)

Adjusting Coefficient: w and Bias/intercept: b

Learnable parameters: w, b

X	Y
5	40
7	120
12	180
16	210
20	240



In order to train a linear regression model, we need to define a cost function and an optimizer.

The cost function is used to measure how well our model fits the data, while the optimizer decides which direction to move in order to improve this fit.

Cost function: MSE Loss - Mean Squared Error

- $MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$
 - $\hat{y}$: prediction
 - y : true value

Optimizer: Gradient Descent

- Simplified equation
 - $\theta = \theta - \eta \cdot \nabla_{\theta}$
 - θ : parameters (our variables)
 - η : learning rate (how fast we want to learn)
 - ∇_{θ} : parameters' gradients
- Even simpler equation
 - `parameters = parameters - learning_rate * parameters_gradients`

Problem 1: Building a Linear Regression Model

For the following training data, build a regression model. Assume w and b is initialized with 1 and learning parameter is set to 0.001.

```
x = torch.tensor([5.0, 7.0, 12.0, 16.0, 20.0])  
y = torch.tensor([40.0, 120.0, 180.0, 210.0, 240.0])
```

```
import torch
from matplotlib import pyplot as plt

# Create the tensors x and y. They are the training
# examples in the dataset for the linear regression
x = torch.tensor(
    [12.4, 14.3, 14.5, 14.9, 16.1, 16.9, 16.5, 15.4, 17.0, 17.9, 18.8, 20.3, 22.4, 19.4, 15.5, 16.7, 17.3, 18.4, 19.2,
     17.4, 19.5, 19.7, 21.2])
y = torch.tensor(
    [11.2, 12.5, 12.7, 13.1, 14.1, 14.8, 14.4, 13.4, 14.9, 15.6, 16.4, 17.7, 19.6, 16.9, 14.0, 14.6, 15.1, 16.1, 16.8,
     15.2, 17.0, 17.2, 18.6])

# The parameters to be Learnt w, and b in the
# prediction  $y_p = wx + b$ 
b = torch.rand([1], requires_grad=True)
w = torch.rand([1], requires_grad=True)
print("The parameters are {}, and {}".format(w, b))

# The Learning rate is set to  $\alpha = 0.001$ 
learning_rate = torch.tensor(0.001)

#The List of Loss values for the plotting purpose
loss_list = []
```

```

# Run the training loop for N epochs
for epochs in range(100):
    #Compute the average Loss for the training samples
    loss = 0.0
    #Accumulate the Loss for all the samples
    for j in range(len(x)):
        a = w * x[j]
        y_p = a + b
        loss += ( y_p-y[j]) ** 2
    #find the average Loss
    loss = loss / len(x)
    #Add the Loss to a list for the plotting purpose
    loss_list.append(loss.item())
    #Compute the gradients using backward
    # dL/dw and dL/db
    loss.backward()
    # Without modifying the gradient in this block
    # perform the operation
    with torch.no_grad():
        # Update the weight based on gradient descent
        # equivalently one may write w1.copy_(w1 - Learning_rate * w1.grad)
        w -= learning_rate * w.grad
        b -= learning_rate * b.grad
    #reset the gradients for next epoch
    w.grad.zero_()
    b.grad.zero_()
    #w.grad = None
    #b.grad = None
    # prev_Loss = Loss
    #Display the parameters and Loss
    print("The parameters are w={}, b={}, and loss={}".format(w, b, loss.item()))
#Display the plot
plt.plot(loss_list)
plt.show()

```

Exercise Questions:

1. For the following training data, build a linear regression model. Assume w and b are initialized with 1 and learning parameter is set to 0.001.

```
x = torch.tensor( [12.4, 14.3, 14.5, 14.9, 16.1, 16.9, 16.5, 15.4, 17.0, 17.9, 18.8, 20.3, 22.4, 19.4, 15.5, 16.7, 17.3, 18.4, 19.2, 17.4, 19.5, 19.7, 21.2])
```

```
y = torch.tensor( [11.2, 12.5, 12.7, 13.1, 14.1, 14.8, 14.4, 13.4, 14.9, 15.6, 16.4, 17.7, 19.6, 16.9, 14.0, 14.6, 15.1, 16.1, 16.8, 15.2, 17.0, 17.2, 18.6])
```

Assume learning rate = 0.001. Plot the graph of epoch in x axis and loss in y axis.

2. Find the value of $w.grad$, $b.grad$ using analytical solution for the given linear regression problem. Initial value of $w = b = 1$. Learning parameter is set to 0.001. Implement the same and verify the values of $w.grad$, $b.grad$ and updated parameter values for two epochs. Consider the difference between predicted and target values of y is defined as $(yp - y)$.

x	y
2	20
4	40

3. Revise the linear regression model by defining a user defined class titled `RegressionModel` with two parameters w and b as its member variables. Define a constructor to initialize w and b with value 1. Define four member functions namely `forward(x)` to implement $wx + b$, `update()` to update w and b values, `reset_grad()` to reset parameters to zero, `criterion(y, yp)` to implement MSE Loss given the predicted y value yp and the target label y . Define an object of this class named *model* and invoke all the methods. Plot the graph of epoch vs loss by varying epoch to 100 iterations.

```
x = torch.tensor([5.0, 7.0, 12.0, 16.0, 20.0])
```

```
y = torch.tensor([40.0, 120.0, 180.0, 210.0, 240.0])
```

```
learning_rate = torch.tensor(0.001)
```



```

class RegressionModel:
    def __init__(self):
        self.w = torch.rand([1], requires_grad=True)
        self.b = torch.rand([1], requires_grad=True)
    def forward(self, x):
        return self.w * x + self.b
    def update(self):
        # Update the weight based on gradient descent
        # equivalently one may write w1.copy_(w1 - learning_rate * w1.grad)
        self.w -= learning_rate * self.w.grad
        self.b -= learning_rate * self.b.grad

    def reset_grad(self):
        self.w.grad.zero_()
        self.b.grad.zero_()

def criterion(yj, y_p):
    return (yj - y_p)**2

```

```

model = RegressionModel()

#The list of loss values for the plotting purpose
loss_list = []

# Run the training loop for N epochs
for epochs in range(100):
    loss = 0.0
    #Accumulate the loss for all the samples
    for j in range(len(x)):
        y_p = model.forward(x[j])
        loss += criterion(y[j], y_p)

    #Find the average loss
    loss = loss / len(x)
    #Add the loss to a list for the plotting purpose
    loss_list.append(loss.item())

```

```

#Compute the gradients using backward dL/dw and dL/db
loss.backward()

# Without modifying the gradient in this block
# perform the operation
with torch.no_grad():
    model.update()
#reset the gradients for next epoch
model.reset_grad()

#w.grad = None
#b.grad = None

# prev_loss = loss
#Display the parameters and loss
print("The parameters are w={}, b={}, and loss={}".format(model.w, model.b, loss.item()))

#Display the plot
plt.plot(loss_list)
plt.show()

```

4. Convert your program written in Qn 3 to extend nn.module in your model. Also override the necessary methods to fit the regression line. Illustrate the use of Dataset and DataLoader from torch.utils.data in your implementation. Use the SGD Optimizer torch.optim.SGD()
5. Use PyTorch's nn.Linear() in your implementation to perform linear regression for the data provided in Qn. 1. Also plot the graph.
6. Implement multiple linear regression for the data provided below

Subject	X1	X2	Y
1	3	8	-3.7
2	4	5	3.5
3	5	7	2.5
4	6	3	11.5
5	2	1	5.7

Verify your answer for the data point $X_1=3$, $X_2=2$.

7. Implement logistic regression

$x = [1, 5, 10, 10, 25, 50, 70, 75, 100,]$

$y = [0, 0, 0, 0, 0, 1, 1, 1, 1]$

Additional Question:

1. Find the value of $w.grad$, $b.grad$ using analytical solution for the given linear regression problem. Initial value of $w = b = 1$. Learning parameter is set to 0.001. Implement the same and verify the values of $w.grad$, $b.grad$ and updated parameter values for two epochs.

Consider the difference between predicted and target values of y is defined as $(y - y_p)$.

x	y
2	20
4	40

Feedforward Neural Network

Objectives:

In this lab, student will be able to

- To build single and multi-layered neural networks
- Develop PyTorch deep learning models for tasks such as XOR problem and MNIST digit classification.

A feedforward neural network is a type of artificial neural network in which nodes' connections do not form a loop. The feed forward model is a basic type of neural network because the input is only processed in one direction.

Often referred to as a multi-layered network of neurons, feedforward neural networks are so named because all information flows in a forward manner only.

The data enters the input nodes, travels through the hidden layers, and eventually exits the output nodes. The network is devoid of links that would allow the information exiting the output node to be sent back into the network.

A Feedforward Neural Network Layers

The following are the components of a feedforward neural network:

Layer of input

It contains the neurons that receive input. The data is subsequently passed on to the next tier. The input layer's total number of neurons is equal to the number of variables in the dataset.

Hidden layer

This is the intermediate layer, which is concealed between the input and output layers. This layer has a large number of neurons that perform alterations on the inputs. They then communicate with the output layer. These layers use activation functions, such as ReLU or

sigmoid, to introduce non-linearity into the network, allowing it to learn and model more complex relationships between the inputs and outputs.

Output layer

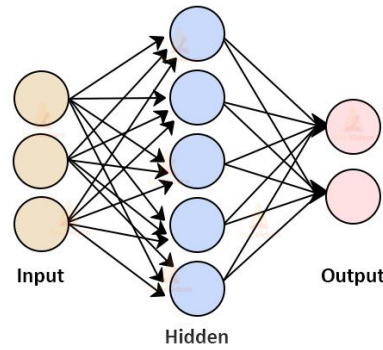
It is the last layer and is depending on the model's construction. Additionally, the output layer is the expected feature or the desired outcome. The output layer generates the final output. Depending on the type of problem, the number of neurons in the output layer may vary. For example, in a binary classification problem, it would only have one neuron. In contrast, a multi-class classification problem would have as many neurons as the number of classes.

Neurons weights

Weights are used to describe the strength of a connection between neurons. The range of a weight's value is from 0 to 1.

When there is a non-linear and complex relationship between X and Y, nevertheless, a Linear Regression method may struggle to predict Y. To approximate that relationship, we may need a curve or a multi-dimensional curve in this scenario.

Neural Network Architecture



A weight is being applied to each input to an artificial neuron. First, the inputs are multiplied by their weights, and then a bias is applied to the outcome. This is called the weighted sum. After that, the weighted sum is processed via an activation function, as a non-linear function.

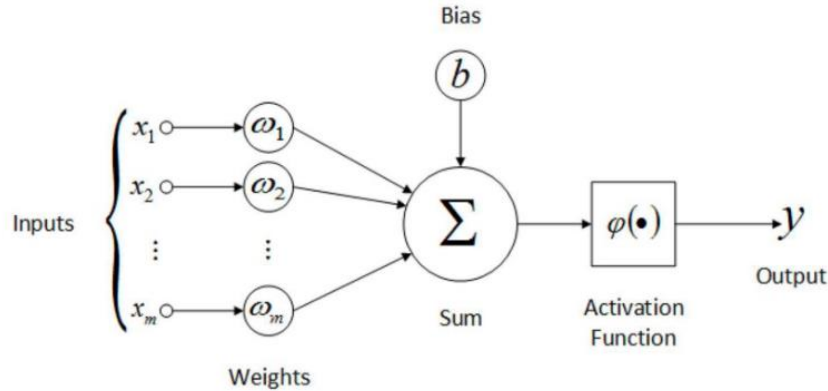
The first layer is the input layer, which appears to have six neurons but is only the data that is sent into the neural network. The output layer is the final layer. The dataset and the type of challenge determine the number of neurons in the final layer and the first layer. Trial and error will be used to determine the number of neurons in the hidden layers and the number of hidden layers.

All of the inputs from the previous layer will be connected to the first neuron from the first hidden layer. The second neuron in the first hidden layer will be connected to all of the preceding layer's inputs, and so forth for all of the first hidden layer's neurons. The outputs of the previously hidden layer are regarded inputs for neurons in the second hidden layer, and each of these neurons is coupled to all of the preceding neurons.

The number of neurons and layers in the hidden layers is one of the hyperparameters that can be adjusted during the design and training of the network. Generally speaking, the more neurons and layers there are, the more complex and abstract features the network can learn. However, this also increases the risk of overfitting and requires more computational power to train the network.

Single Layer feed forward network:

In its most basic form, a Feed-Forward Neural Network is a single layer perceptron. A sequence of inputs enter the layer and are multiplied by the weights in this model. The weighted input values are then summed together to form a total. If the sum of the values is more than a predetermined threshold, which is normally set at zero, the output value is usually 1, and if the sum is less than the threshold, the output value is usually -1. The single-layer perceptron is a popular feed-forward neural network model that is frequently used for classification.



The neural network can compare the outputs of its nodes with the desired values using a property known as the delta rule, allowing the network to alter its weights through training to create more accurate output values. This training and learning procedure results in gradient descent.

Importance of the Non-Linearity

When two or more linear objects, such as a line, plane, or hyperplane, are combined, the outcome is also a linear object: line, plane, or hyperplane. No matter how many of these linear things we add, we'll still end up with a linear object.

However, this is not the case when adding non-linear objects. When two separate curves are combined, the result is likely to be a more complex curve.

We're introducing non-linearity at every layer using these activation functions, in addition to just adding non-linear objects or hyper-curves like hyperplanes. In other words, we're applying a nonlinear function on an already nonlinear object.

What if activation functions were not used in neural networks?

Suppose if neural networks didn't have an activation function, they'd just be a huge linear unit that a single linear regression model could easily replace.

$$a = m \cdot x + d$$

$$Z = k \cdot a + t \Rightarrow k \cdot (m \cdot x + d) + t \Rightarrow k \cdot m \cdot x + k \cdot d + t \Rightarrow (k \cdot m) \cdot x + (k \cdot d + t)$$

Activation Function

An activation function is a mathematical function applied to a neuron's output in a neural network feedforward. It introduces non-linearity into the network, allowing it to learn and model more complex relationships between the inputs and outputs. Without the activation function, a neural network would be linear, less powerful, and less expressive than a non-linear model.

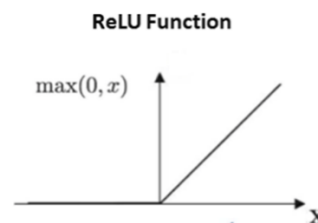
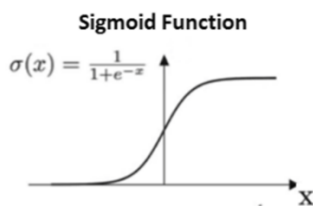
There are many different activation functions that we can use in a neural networks feedforward; some of the most common ones include the following:

Sigmoid:

The sigmoid activation function maps any input value to a value between 0 and 1, which is useful for binary classification problems.

The rectified linear unit (ReLU):

It is a popular choice in neural networks. It is defined as $f(x)=\max(0,x)$, where x is the input to the function. For any input x , the output of the ReLU function is x if x is positive and 0 if x is negative. This activation function is simple to implement computationally and faster than other non-linear activation function like tanh or sigmoid.

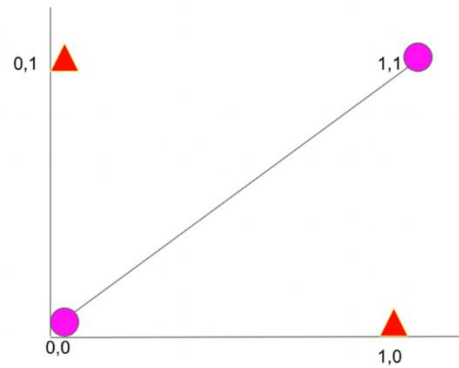


When our data is not linear separable, linear models face problems in approximating whereas it is easy for the neural networks. The hidden layers are used to increase the non-linearity and change the representation of the data for better generalization over the function.

Exercise Questions:

1. Implement two layer Feed Forward Neural Network for XOR Logic Gate with 2-bit Binary Input using Sigmoid activation. Verify the number of learnable parameters in the model.

X_1	X_2	$Y = X_1 \text{ XOR } X_2$
0	0	0
0	1	1
1	0	1
1	1	0



Define the neural network model

Import necessary Libraries

```
import torch
from matplotlib import pyplot as plt
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import numpy as np
```

```
loss_list = []
torch.manual_seed(42)
```

Step 1: Initialize inputs and expected outputs as per the truth table of XOR

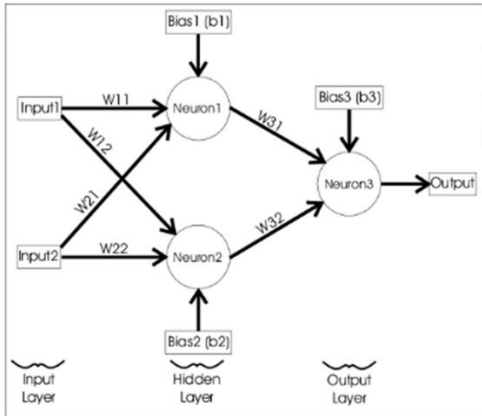
Create the tensors x_1, x_2 and y .

They are the training examples in the dataset for the XOR

```
X = torch.tensor([[0,0],[0,1],[1,0],[1,1]], dtype=torch.float32)
```

```
Y = torch.tensor([0,1,1,0], dtype=torch.float32)
```

Step 2: Define XORModel class - write constructor and forward function



```
class XORModel(nn.Module):
    def __init__(self):
        super(XORModel, self).__init__()
        #self.w = torch.nn.Parameter(torch.rand([1]))
        #self.b = torch.nn.Parameter(torch.rand([1]))

        self.linear1 = nn.Linear(2,2,bias=True)
        self.activation1 = nn.Sigmoid()
        self.linear2 = nn.Linear(2,1,bias=True)
        #self.activation2 = nn.ReLU()

    def forward(self, x):
        x = self.linear1(x)
        x = self.activation1(x)
        x = self.linear2(x)
        #x = self.activation2(x)
        return x
```

Step 3: Create DataLoader. Write Dataset class with necessary constructors and methods – len() and getitem()

```
class MyDataset(Dataset):
    def __init__(self, X, Y):
        self.X = X
        self.Y = Y

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx].to(device), self.Y[idx].to(device)
```

#Create the dataset

full_dataset = MyDataset(X, Y)

batch_size = 1

#Create the dataloaders for reading data - # This provides a way to read the dataset in batches, also shuffle the data

train_data_loader = DataLoader(full_dataset, batch_size=batch_size, shuffle=True)

#Find if CUDA is available to load the model and device on to the available device CPU/GPU

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

#Load the model to GPU

model = XORModel().to(device)

print(model)

#Add the criterion which is the MSELoss

loss_fn = torch.nn.MSELoss()

#Optimizers specified in the torch.optim package

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.03)
```

```
EPOCHS = 10000
for epoch in range(EPOCHS):
    #print('EPOCH {}'.format(epoch + 1))

    # Make sure gradient tracking is on, and do a pass over the data
    model.train(True)
    avg_loss = train_one_epoch(epoch)
    loss_list.append(avg_loss)
    #loss_list.append(avg_loss.detach().cpu())

    #print('LOSS train {}'.format(avg_loss))

    if epoch % 1000 == 0:
        print(f'Epoch {epoch}/{EPOCHS}, Loss: {avg_loss}')
```

Training an epoch

```
def train_one_epoch(epoch_index):
    totalloss = 0.
    # use enumerate(training_loader) instead of iter
    for i, data in enumerate(train_data_loader):
        # Every data instance is an input + label pair
        inputs, labels = data

        # Zero your gradients for every batch!
        optimizer.zero_grad()

        # Make predictions for this batch
        outputs = model(inputs)

        # Compute the Loss and its gradients
        loss = loss_fn(outputs.flatten(), labels)
        loss.backward()

        # Adjust Learning weights
        optimizer.step()

        # Gather data and report
        totalloss += loss.item()

    return totalloss/(len(train_data_loader) * batch_size)
```

Model Inference step

```

for param in model.named_parameters():
    print(param)
Model inference – similar to prediction in ML
input = torch.tensor([0, 1], dtype=torch.float32).to(device)
model.eval()
print("The input is = {}".format(input))
print("Output y predicted = {}".format(model(input)))
#Display the plot
plt.plot(loss_list)
plt.show()

```

Output – Model parameters

```

Model parameters= ('linear1.weight', Parameter containing:
tensor([[ 0.5413, 0.5890],
        [-0.1679, 0.6455]], requires_grad=True))
Model parameters= ('linear1.bias', Parameter containing:
tensor([-0.1505, 0.1357], requires_grad=True))
Model parameters= ('linear2.weight', Parameter containing:
tensor([[ -0.3832, 0.3738]], requires_grad=True))
Model parameters= ('linear2.bias', Parameter containing:
tensor([0.5562], requires_grad=True))

```

2. Repeat Qn 1 by modifying the activation function to ReLU.
3. Manually verify the output values by taking system generated values of weights and biases for both Linear1 and Linear2 layers for Qn 1 and apply the transformations to input X and implement the same.
4. Implement Feed Forward Neural Network with two hidden layers for classifying handwritten digits using MNIST dataset. Display the classification accuracy in the form of a Confusion matrix. Verify the number of learnable parameters in the model.

Convolution Neural Networks

Objectives:

In this lab, student will be able to

- Perform convolution operation on images
- To build convolutional neural network (CNN)
- Develop a CNN for MNIST digit classification using PyTorch library.

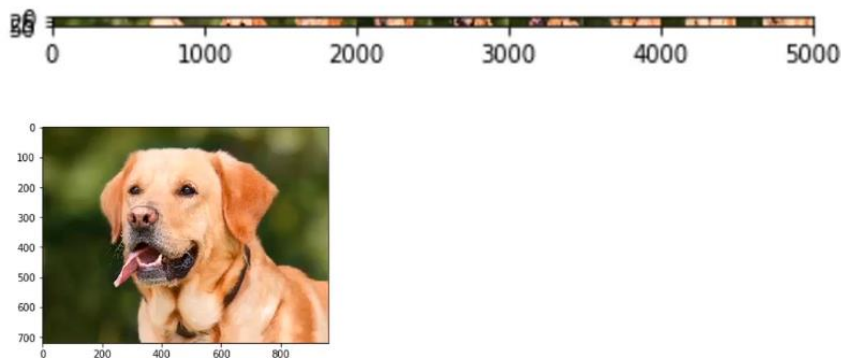
We can consider Convolutional Neural Networks, or CNNs, as feature extractors that help to extract features from images.

In a simple neural network, we convert a 3-dimensional image to a single dimension. Following image is a 1-D representation whereas the second one is a 2-D representation of the same image.

Spatial Orientation

Here, the orientation of the images has been changed but we are unable to identify it by looking at the 1-D representation.

This is the problem with artificial neural networks – they lose spatial orientation



Large number of parameters

Another problem with neural networks is the large number of parameters. If our image has a size of $28 \times 28 \times 3$ – so the parameters here will be 2,352. What if we have an image of size $224 \times 224 \times 3$? The number of parameters here will be 150,528. And these parameters will only increase as we increase the number of hidden layers.

So, the two major disadvantages of using artificial neural networks are:

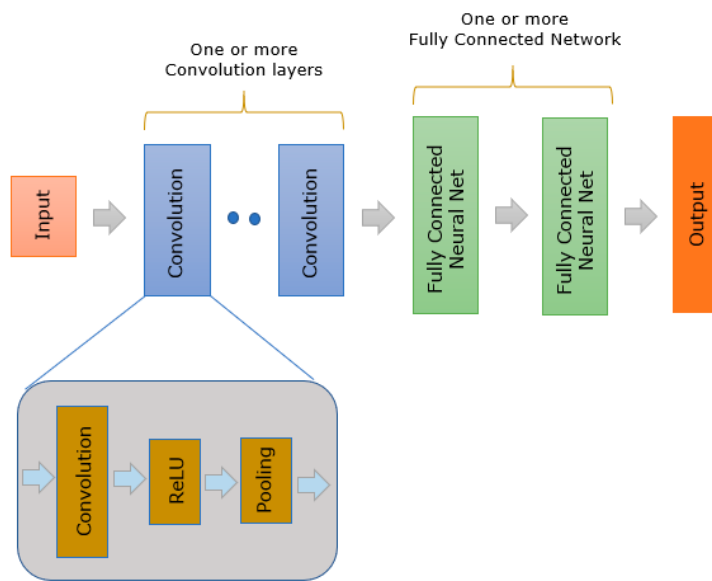
1. Loses spatial orientation of the image

2. The number of parameters increases drastically

How can we preserve the spatial orientation as well as reduce the learnable parameters. CNNs help to extract features from the images which may be helpful in classifying the objects in that image. It starts by extracting low dimensional features (like edges) from the image, and then some high dimensional features like the shapes. We use filters to extract features from the images and Pooling techniques to reduce the number of learnable parameters.

CNN (Convolutional Neural Network) Architecture:

CNN is made up two large groups of components :: Convolutional Layer and Fully Connected Network layer.



Convolutional Layer : This performs the function of pattern detection from the input image. The pattern detection by convolution layer is proved working very well even when the size of the target image varies (changes), position and rotation of the image changes. It would be easy to realize that this would be much better than hand-written rule based pattern detection. In reality, it works better than fully connected network. It is said that we may need much more neurons (i.e, much more number of weights) for a fully connected network to perform the same level as convolutional layers. On top of it, Fully Connected Network may not be as robust as convolutional layer in terms of finding patterns from geometrically transformed image (like different size, rotated, translated).

Fully Connected Network : This performs the classification of the image combining a different types of patterns detected by Convolutional Layers.

The result of the convolution goes through a special function called ReLU(Rectified Linear Unit). ReLU is a activation function(transfer function) for Convolution layer. The output of ReLU data

goes through another process called Pooling. Pooling is a kind of Sampling method to reduce the number of data without losing the critical nature of the convolved data.

After the convolution layer in CNN, there exist one or more of fully connected neural network. Fully Connected Network is a simple feedforward network.

In CNN, the exact type of features to be detected is determined by the network itself during the learning process. The element values for each of the kernel is not fixed/predefined and is set randomly and gets updated (changes) during learning process.

CNN has become one of the most popular neural networks since it showed such a great performance on image classification. Most of neural network that is used for image recognition or classification is based on CNN.

Exercise Questions:

1. Implement convolution operation for a sample image of shape (H=6, W=6, C=1) with a random kernel of size (3,3) using torch.nn.functional.conv2d.

```
import torch
import torch.nn.functional as F
image = torch.rand(6,6)
print("image=", image)
#Add a new dimension along 0th dimension
#i.e. (6,6) becomes (1,6,6). This is because
#pytorch expects the input to conv2D as 4d tensor
image = image.unsqueeze(dim=0)
print("image.shape=", image.shape)
image = image.unsqueeze(dim=0)
print("image.shape=", image.shape)
print("image=", image)
kernel = torch.ones(3,3)
#kernel = torch.rand(3,3)
print("kernel=", kernel)
kernel = kernel.unsqueeze(dim=0)
kernel = kernel.unsqueeze(dim=0)
#Perform the convolution
outimage = F.conv2d(image, kernel, stride=1, padding=0)
print("outimage=", outimage)
```

What is the dimension of the output image? Apply, various values for parameter *stride=1* and note the change in the dimension of the output image. Arrive at an equation for the output image size with respect to the kernel size and stride and verify your answer with

code. Now, repeat the exercise by changing padding parameter. Obtain a formula using kernel, stride, and padding to get the output image size. What is the total number of parameters in your network? Verify with code.

2. Apply `torch.nn.Conv2d` to the input image of Qn 1 with `out-channel=3` and observe the output. Implement the equivalent of `torch.nn.Conv2d` using the `torch.nn.functional.conv2D` to get the same output. You may ignore bias.
3. Implement CNN for classifying digits in MNIST dataset using PyTorch. Display the classification accuracy in the form of a Confusion matrix. Verify the number of learnable parameters in the model.

Training a CNN on an image dataset is similar to training a basic multi-layer feed-forward network on numerical data as outlined below.

Define model architecture

Load dataset from disk

Loop over epochs and batches

Make predictions and compute loss

Properly zero our gradient, perform backpropagation, and update model parameters

```
class CNNClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        #self.w = torch.nn.Parameter(torch.rand([1]))
        #self.b = torch.nn.Parameter(torch.rand([1]))
        self.net = nn.Sequential(nn.Conv2d(1, 64, kernel_size=3),
                                nn.ReLU(),
                                nn.MaxPool2d((2, 2), stride=2),
                                nn.Conv2d(64, 128, kernel_size=3),
                                nn.ReLU(),
                                nn.MaxPool2d((2, 2), stride=2),
                                nn.Conv2d(128, 64, kernel_size=3),
                                nn.ReLU(),
                                nn.MaxPool2d((2, 2), stride=2)
                                )
        self.classification_head = nn.Sequential(nn.Linear(64, 20, bias=True),
                                                  nn.ReLU(),
                                                  nn.Linear(20, 10, bias=True))

    def forward(self, x):
        features = self.net(x)
        return self.classification_head(features.view(batch_size, -1))
```

4. Modify CNN of Qn. 3 to reduce the number of parameters in the network. Draw a plot of percentage drop in parameters vs accuracy.

Additional Question:

1. Design CNN to classify images using Fashion MNIST dataset. Display the classification accuracy in the form of a Confusion matrix. Verify the number of learnable parameters in the model.

Transfer Learning

Objectives:

In this lab, student will be able to

1. Apply the concept of transfer learning
2. Comprehend transfer learning via feature extraction and transfer learning via fine tuning
3. Use well-known pretrained models in the implementation

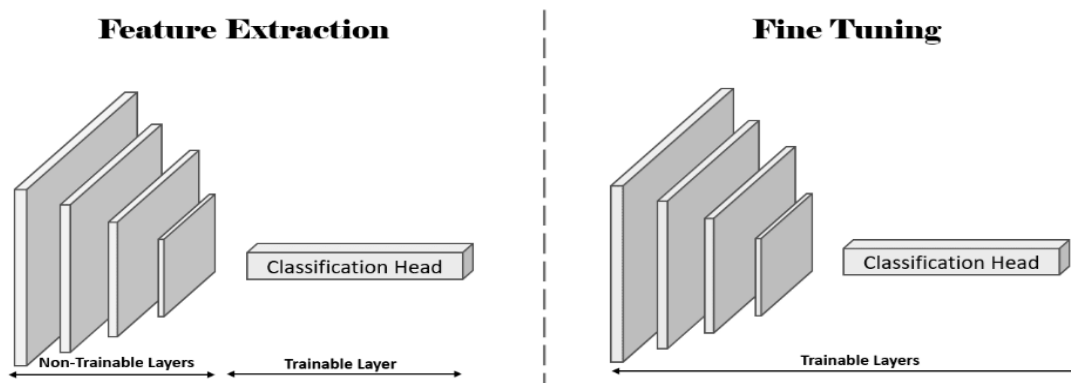
Transfer learning is a machine learning technique where a model trained on one task is adapted for a second related task. In other words, knowledge gained from solving one problem is applied to a different but related problem. This approach is particularly useful when there is a limited amount of labelled data available for the task at hand.

In traditional machine learning, models are trained to perform a specific task, and they don't generalize well to new, unseen tasks. Transfer learning aims to overcome this limitation by leveraging knowledge gained from a source task to improve learning in a target task.

There are different strategies for transfer learning, but they generally fall into the following categories:

Fine-Tuning: Instead of freezing the earlier layers, the entire pre-trained model is fine-tuned on the target task. This is common when the source and target tasks are more closely related, and the entire model needs to be adapted to the specifics of the new task.

Feature Extraction: In this approach, a pre-trained model is used as a fixed feature extractor. The earlier layers of the model, which are usually responsible for detecting low-level features, are frozen, and only the later layers are re-trained on the target task. This is useful when the source and target tasks share similar low-level features.



Transfer learning is widely used in deep learning, especially with convolutional neural networks (CNNs) for computer vision tasks and recurrent neural networks (RNNs) for natural language

processing tasks. Pre-trained models, such as those trained on ImageNet for image classification, BERT for natural language processing, or GPT for generative tasks, have been successfully applied in various domains using transfer learning.

Transfer learning is used for several reasons, and it offers several advantages in the field of machine learning and deep learning:

Limited Data Availability: Transfer learning is especially useful when there is limited labelled data available for the specific task at hand. Training a deep learning model from scratch often requires large amounts of data, but pre-trained models can be fine-tuned on a smaller dataset, leveraging knowledge gained from a larger, related dataset.

Computational Efficiency: Training deep learning models can be computationally expensive and time-consuming. Transfer learning allows practitioners to use pre-trained models as a starting point, reducing the computational resources needed to train a model for a new task.

Feature Extraction: Pre-trained models, particularly those trained on massive datasets, have learned rich feature representations that can be useful across different tasks. Transfer learning allows practitioners to use these pre-learned features as a starting point for a new task, potentially improving performance.

Domain Adaptation: Transfer learning is valuable when there is a shift in the distribution of the data between the source and target tasks. By leveraging knowledge from a source domain, the model can adapt more quickly to the target domain.

Task Similarity: Transfer learning is most effective when the source and target tasks are related or have some underlying similarity. If the lower-level features are applicable to both tasks, then the knowledge gained from the source task can be beneficial for the target task.

Model Generalization: Transfer learning helps improve the generalization capabilities of a model. Instead of starting from scratch, the model begins with knowledge gained from solving a different but related problem, which can lead to better performance on the target task.

Popular applications of transfer learning include computer vision tasks (using pre-trained models for image classification or object detection), natural language processing tasks (using pre-trained language models for sentiment analysis or named entity recognition), and more.

Overall, transfer learning provides a practical way to leverage existing knowledge and resources, making it a valuable tool in scenarios where data or computational resources are limited.

Exercises:

1. Perform classification on FashionMNIST, fashion apparels dataset, using a pre-trained model which is trained on MNIST handwritten digit classification dataset.

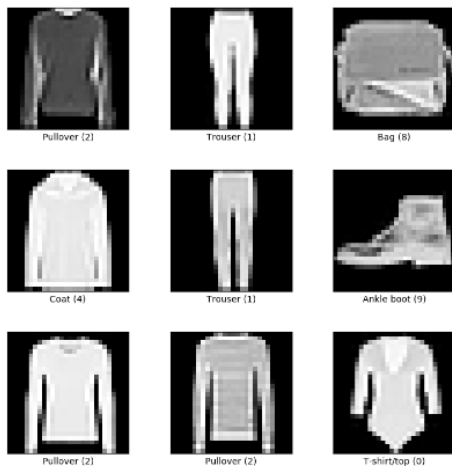
Fashion-MNIST is a dataset comprising of 28×28 grayscale images of 70,000 fashion products from 10 categories, with 7,000 images per category. The training set has 60,000 images and the test set has 10,000 images. Fashion-MNIST shares the same image size, data format and the structure of training and testing splits with the original MNIST.

Split	Examples
'test'	10,000
'train'	60,000

- **Feature structure:**

```
FeaturesDict({  
    'image': Image(shape=(28, 28, 1), dtype=uint8),  
    'label': ClassLabel(shape=(), dtype=int64, num_classes=10),  
})
```

Display examples are shown below.



Creating the pre-trained model (MNIST_CNN.py): original source program

Use state_dict to save model parameters and Optimizer information.

```

loss_fn = torch.nn.CrossEntropyLoss()

# Optimizers specified in the torch.optim package
optimizer = torch.optim.SGD(model.parameters(), lr=0.001)

# Print model's state_dict
print("Model's state_dict:")
for param_tensor in model.state_dict().keys():
    print(param_tensor, "\t", model.state_dict()[param_tensor].size())
print()

# Print optimizer's state_dict
print("Optimizer's state_dict:")
for var_name in optimizer.state_dict():
    print(var_name, "\t", optimizer.state_dict()[var_name])

EPOCHS = 2
for epoch in range(EPOCHS):
    print('EPOCH {}:'.format(epoch + 1))

```

Step 1: Re-run the MNIST program by appending the command `torch.save(model, “./ModelFiles/model.pt”)` at the end. Make sure **ModelFiles** folder exists in the current working directory.

```
torch.save(model, “./ModelFiles/model.pt”)
```

Using the pretrained model for inference (**FashionMNIST_CNN.py**):

Step 2: Import libraries same as in `MNIST_CNN.py`

Step 3: Define the class with same as in `MNIST_CNN.py`

Step 4: There is no need to train the model hence use testloader.

```

mnist_testset = datasets.FashionMNIST(root='./data', train=False,
download=True, transform = ToTensor())

test_loader = DataLoader(mnist_testset, batch_size=batch_size, shuffle=False)

```

step 5: Load the pretrained model on to the device

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Load the model to GPU

```

model = CNNClassifier()
model = torch.load("./ModelFiles/model.pt")
model.to(device)

```

Step 6: Print the model state dictionary. (Same can be done in MNIST_CNN.py)

```
# Print model's state_dict. We are printing only the size of the parameter
print("Model's state_dict:")
for param_tensor in model.state_dict().keys():
    print(param_tensor, "\t", model.state_dict()[param_tensor].size())
print()
```

Step 7: Evaluate the model

```
model.eval()

correct = 0
total = 0
for i, vdata in enumerate(test_loader):
    tinputs, tlabels = vdata
    tinputs = tinputs.to(device)
    tlabels = tlabels.to(device)
    toutputs = model(tinputs)
    #Select the predicted class label which has the
    # highest value in the output layer
    _, predicted = torch.max(toutputs, 1)
    print("True label:{}".format(tlabels))
    print('Predicted: {}'.format(predicted))
    # Total number of labels
    total += tlabels.size(0)

    # Total correct predictions
    correct += (predicted == tlabels).sum()

accuracy = 100.0 * correct / total
print("The overall accuracy is {}".format(accuracy))
```

2. Learn the AlexNet architecture and apply transfer learning to perform the classification task. Using the pre-trained AlexNet, classify images from the cats_and_dogs_filtered dataset [downloaded](#) from the below link. Finetune the classifier given in AlexNet as a two-class classifier. Perform pre-processing of images as per the requirement.

https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip

The "cats_and_dogs_filtered" dataset is a subset of the larger Dogs vs. Cats dataset used for binary classification tasks. It contains images of cats and dogs, each belonging to one of the two classes. The subset includes a training set and a validation set. cats_and_dogs_filtered dataset consists of train and validation sets of 1000 and 500 items of cats and dogs images respectively. Key features of dataset are listed below:

Dataset Structure:

The dataset consists of separate folders for training and validation. Each class (cats and dogs) has its subfolder containing images of that class.

Image Preprocessing:

Common preprocessing steps include resizing images to a standard size, normalizing pixel values, and data augmentation (such as random cropping and flipping) to improve the model's generalization.

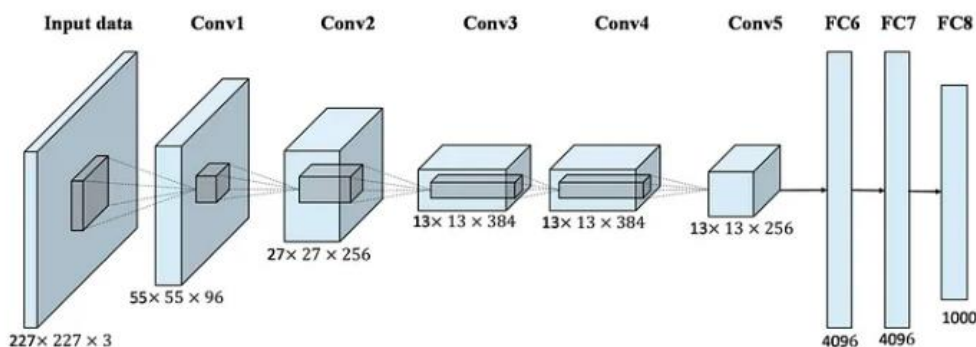
Task:

The primary task associated with this dataset is binary classification – distinguishing between images of cats and dogs

AlexNet is a convolutional neural network (CNN) architecture designed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton. It was the winning architecture in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012, significantly advancing the state-of-the-art in image classification tasks.

It solves the problem of image classification where the input is an image of one of 1000 different classes (e.g. cats, dogs etc.) and the output is a vector of 1000 numbers. The i th element of the output vector is interpreted as the probability that the input image belongs to the i th class. Therefore, the sum of all elements of the output vector is 1.

AlexNet mainly composed of cascaded stages, such as convolution layers, pooling layers, rectified linear unit (ReLU) layers and fully connected layers. It has five convolutional layers and three fully-connected layers.



Key features of AlexNet are listed below:

Architecture:

- AlexNet consists of eight layers, including five convolutional layers and three fully connected layers.
- It uses the Rectified Linear Unit (ReLU) activation function to introduce non-linearity.
- The architecture employs max-pooling layers to downsample spatial dimensions.
- Local Response Normalization (LRN) is applied to enhance the model's generalization ability.

Input Size:

AlexNet takes input images of size $224 \times 224 \times 3$ (RGB channels).

Dropout:

Dropout, a regularization technique, is applied to the fully connected layers to prevent overfitting.

Output:

The final layer is a softmax layer, which produces probabilities for different classes in a multi-class classification problem.

The input to AlexNet is an RGB image of size 256×256 . This means all images in the training set and all test images need to be of size 256×256 . If the input image is not 256×256 , it needs to be converted to 256×256 before using it for training the network. To achieve this, the smaller dimension is resized to 256 and then the resulting image is cropped to obtain a 256×256 image.



If the input image is grayscale, it is converted to an RGB image by replicating the single channel to obtain a 3-channel RGB image. Random crops of size 227×227 were generated from inside the 256×256 images to feed the first layer of AlexNet.

Step 1: Import necessary libraries

```

import PIL.Image
import torch
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
from torchvision import transforms
import glob
from torchvision.models import AlexNet_Weights

```

Step 2: Set AlexNet pretrained to true

```

model = torch.hub.load('pytorch/vision:v0.10.0', model='alexnet', weights=AlexNet_Weights.DEFAULT)
batch_size = 4

# add the criterion which is the MSELoss
loss_fn = torch.nn.CrossEntropyLoss()

# Optimizers specified in the torch.optim package
optimizer = torch.optim.SGD(model.parameters(), lr=0.001)

```

3. Implement check points in PyTorch by saving model state_dict, optimizer state_dict, epochs and loss during training so that the training can be resumed at a later point. Also, illustrate the use of check point to save the best found parameters during training.

Use the original source program **MNIST_CNN_Checkpoint.py**

Use state_dict to save model parameters and Optimizer information.

```

loss_fn = torch.nn.CrossEntropyLoss()

# Optimizers specified in the torch.optim package
optimizer = torch.optim.SGD(model.parameters(), lr=0.001)

# Print model's state_dict
print("Model's state_dict:")
for param_tensor in model.state_dict().keys():
    print(param_tensor, "\t", model.state_dict()[param_tensor].size())
print()

# Print optimizer's state_dict
print("Optimizer's state_dict:")
for var_name in optimizer.state_dict():
    print(var_name, "\t", optimizer.state_dict()[var_name])

EPOCHS = 2
for epoch in range(EPOCHS):
    print('EPOCH {}:'.format(epoch + 1))

```

Step 1: Re-run the MNIST program by appending the following command at the end. Make sure **checkpoints** folder exists in the current working directory.

```

#Save the check point
check_point = {"last_loss":avg_loss, "last_epoch":EPOCHS, "model_state":model.state_dict(),
"optimizer_state":optimizer.state_dict()}
torch.save(check_point, "./checkpoints/checkpoint.pt")

```

Here, a checkpoint has been established soon after two epochs.

Resuming the model for training (**MNIST_CNN_Use_Checkpoint.py**):

The earlier checkpoint is loaded now for resuming the training loop to run for remaining number of epochs as shown below.

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = CNNClassifier().to(device)
#Reload from the checkpoint. This can be done in another file as well.
check_point = torch.load("./checkpoints/checkpoint.pt")
model.load_state_dict(check_point["model_state"])

# add the criterion which is the MSELoss
loss_fn = torch.nn.CrossEntropyLoss()

# Optimizers specified in the torch.optim package
optimizer = torch.optim.SGD(model.parameters(), lr=0.001)

optimizer.load_state_dict(check_point["optimizer_state"])
loss = check_point["last_loss"]
EPOCHS = check_point["last_epoch"]

NEW_EPOCHS = 5
for epoch in range(EPOCHS, NEW_EPOCHS):

```

Additional Exercises:

1. Train a model to classify **ants** and **bees** using the pretrained resnet-18 model. We have approximately 120 training images for both ants and bees, along with 75 validation images for each category. Typically, this dataset is considered quite small for building a robust model from scratch. However, leveraging transfer learning, we anticipate achieving reasonable generalization capabilities. In the solved exercise, we assume that the *train_model* function implemented in earlier lab is reused.

You may download the data from the following link:

https://download.pytorch.org/tutorial/hymenoptera_data.zip

The steps we are going to use for our pre-trained model is:

1. Loading in the pre-trained model
2. Freezing the convolutional layers, when ConvNet is a fixed feature extractor.
3. Replacing the fully connected layers with a custom classifier [Used for fine tuning]
4. Training the custom classifier for the specific task

Regularization

Objectives :

In this lab, student will be able to

1. Address the challenges involved in the training of a deep neural model.
2. Apply regularization such as L1 regularization, L2 regularization to handle overfitting
3. Implement dropout, data augmentation and early stopping

In deep learning, regularization is a set of techniques used to prevent overfitting and improve the generalization performance of a model. Overfitting occurs when a model learns the training data too well, including its noise and outliers, to the extent that it performs poorly on unseen or new data. Regularization methods aim to guide the learning process, preventing the model from becoming too complex and capturing noise in the training data.

Some of the common regularization techniques used in deep learning are: **both add penalty term**

L1 Regularization (Lasso): This technique adds a penalty term to the loss function proportional to the absolute values of the model parameters. It encourages sparsity in the model by pushing some of the parameters to exactly zero.

L2 Regularization (Ridge): L2 regularization adds a penalty term to the loss function proportional to the square of the model parameters. It discourages large weights and helps prevent overfitting by promoting a more balanced distribution of weights.

Dropout: Dropout is a technique where randomly selected neurons are ignored during training. This helps prevent co-adaptation of neurons and makes the model more robust. During testing, all neurons are used, but their weights are scaled to account for the dropout during training.

Data Augmentation: Regularization can also be achieved through data augmentation, which involves applying random transformations to the training data (e.g., rotation, scaling, flipping). This artificially increases the size of the training dataset and makes the model more invariant to variations in the input data.

Early Stopping: Monitor the model's performance on a validation set during training. If the performance stops improving or starts to degrade, training is stopped early to prevent overfitting.

Early stopping is a regularization technique commonly used in deep learning to prevent overfitting and improve the generalization performance of a model. The basic idea behind early stopping is to monitor the performance of the model on a validation dataset during training and stop the training process once the performance starts to degrade, indicating that the model is starting to overfit the training data.

The typical procedure for early stopping involves the following steps:

Training Monitoring: As the model is trained on the training dataset, its performance is periodically evaluated on a separate validation dataset.

Validation Performance Check: The performance metric (such as accuracy, loss, or other relevant metrics) on the validation dataset is monitored. If the performance on the validation set stops improving or begins to degrade after an initial improvement, it may be an indication that the model is overfitting.

Stopping Criteria: A stopping criterion is defined, such as observing no improvement in the validation metric for a specified number of consecutive epochs.

Early Stopping Decision: If the stopping criterion is met, the training process is halted, and the model parameters from the epoch with the best validation performance are retained.

Early stopping helps prevent the model from learning the noise in the training data and encourages the model to generalize well to unseen data. It is particularly useful when training deep neural networks that have a large number of parameters, as these models are prone to overfitting.

Sample Exercise:

Explore the regularization effects of data augmentation. Use image data “cats_and_dogs_filtered” and apply random transformations (e.g., rotations, flips, and noise) to artificially increase the size of the training dataset. Compare the model's performance with and without data augmentation.

The following example shows how we can add a gaussian noise to data as data augmentation. We define a class Gaussian for adding the noise with the specified mean and variance. This is included in the composite transformation using Compose() method defined in the torchvision.transform.

```
import PIL
import torch
import torchvision.transforms as T
from PIL import Image
import glob
from torch.utils.data import Dataset, DataLoader
```

```

from matplotlib import pyplot as plt

img = Image.open('sample.jpg')

class Gaussian(object):
    def __init__(self, mean: float, var: float):
        self.mean = mean
        self.var = var

    def __call__(self, img: torch.Tensor) -> torch.Tensor:
        return img + torch.normal(self.mean, self.var, img.size())

preprocess = T.Compose([
    T.ToTensor(),
    T.RandomHorizontalFlip(),
    T.RandomRotation(45),
    Gaussian(0, 0.15),
])

class MyDataset(Dataset):
    def __init__(self, transform=None, str="train"):
        self.imgs_path = ".\\data\\cats_and_dogs_filtered\\" + str + "\\"
        file_list = glob.glob(self.imgs_path + "*")
        self.data = []
        for class_path in file_list:
            class_name = class_path.split("\\")[-1]
            for img_path in glob.glob(class_path + "\\*.jpg"):
                self.data.append([img_path, class_name])
        self.class_map = {"dogs": 0, "cats": 1}
        self.transform = transform

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        img_path, class_name = self.data[idx]
        img = PIL.Image.open(img_path)
        class_id = self.class_map[class_name]
        class_id = torch.tensor(class_id)
        if self.transform:
            img = self.transform(img)
        return img, class_id

dataset = MyDataset(transform=preprocess, str = "train")
dataloader = DataLoader(dataset)

#The following code displays augmented images
i=0
for data in iter(dataloader):
    img, _ = data

```

```

tI = T.ToPILImage()
img = tI(img.squeeze())
plt.imshow(img)
plt.show()
i = i + 1
if i == 3:
    break

#Extend the code to include training loop

```

Lab Exercises:

1. Implement L2 regularization on cat-dog classification neural network. Train the model on the dataset, and observe the impact of the regularization **on the weight parameters**. (Do not use data augmentation).
 - a. L2 regularization using optimizer's weight decay
 - b. L2 regularization using loop to find L2 norm of weights
2. Implement L1 regularization on cat-dog classification neural network. Train the model on the dataset, and observe the impact of the regularization **on the weight parameters**. (Do not use data augmentation).
 - a. L1 regularization using optimizer's weight decay
 - b. L1 regularization using loop to find L1 norm of weights
3. Implement dropout regularization on cat-dog classification neural network. Train the model with and without dropout on a dataset, and compare the performance and overfitting tendencies.
4. Implement your own version of the dropout layer by using Bernoulli distribution and compare the performance with the library.
5. Implement early stopping as a form of regularization. Train a neural network and monitor the validation loss. Stop training when the validation loss starts increasing, and compare the performance with a model trained without early stopping.

A pseudo-code of the Early stopping algorithm is given below

```

import torch

import torch.nn as nn

import torch.optim as optim

# Define your neural network model

class YourModel(nn.Module):

    # ... model architecture ...

# Initialize your model, loss function, and optimizer

```



```

model = YourModel()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Define early stopping parameters
patience = 5
best_validation_loss = float('inf')
current_patience = 0

# Training loop
for epoch in range(num_epochs):
    # Training steps ...

    # Validation steps
    model.eval()
    with torch.no_grad():
        validation_loss = 0.0
        for inputs, labels in validation_dataloader:
            # Forward pass
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            validation_loss += loss.item()

    # Average validation loss
    validation_loss /= len(validation_dataloader)

    # Check for improvement in validation loss
    if validation_loss < best_validation_loss:

```

```

    best_validation_loss = validation_loss
    current_patience = 0

    # Save the model if desired
    torch.save(model.state_dict(), 'best_model.pth')
else:
    current_patience += 1

    # Check if early stopping criteria are met
    if current_patience > patience:
        print("Early stopping! No improvement for {} epochs.".format(patience))
        break

    # Continue with the next epoch

```

Additional Exercise:

1. Experiment with different values of the regularization strength hyperparameter (e.g., λ in L2 regularization). Train the model with varying regularization strengths and analyze the impact on model performance and generalization.
2. Vary the dropout rate in a neural network and observe the impact on the model's performance. Experiment with different rates and find the optimal dropout rate for your model.
3. Try regularization on ImageNet1K and Animal Face dataset.

Recurrent Neural Network

Objectives :

In this lab, student will be able to

1. Gain hands-on experience by implementing the architecture of RNNs, emphasizing the recurrent connections that allow the network to process sequential information.
2. Apply RNNs to diverse tasks involving sequential data, such as time series prediction, natural language processing,
3. Address vanishing and exploding gradients in RNNs using suitable activation functions

A Recurrent Neural Network (RNN) is a type of artificial neural network designed for processing sequential data and capturing temporal dependencies. Unlike traditional feedforward neural networks, RNNs have connections that form directed cycles, allowing them to maintain a memory of previous inputs in their internal state. This ability to retain information about past inputs makes RNNs well-suited for tasks where context and order are crucial, such as natural language processing, speech recognition, and time series analysis.

The key feature of an RNN is its recurrent connections, which enable information to persist across different time steps. Each step in a sequence involves processing an input, updating the internal state (memory), and producing an output. This recurrent structure allows RNNs to handle variable-length sequences and learn patterns in sequential data.

However, traditional RNNs face challenges such as the vanishing gradient problem, where gradients diminish as they are backpropagated through time, limiting the network's ability to capture long-term dependencies. To address this, more advanced RNN variants, such as Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), have been introduced. These variants incorporate mechanisms to better capture and propagate information over longer sequences. Figure 7.1 shows the architecture of a typical RNN

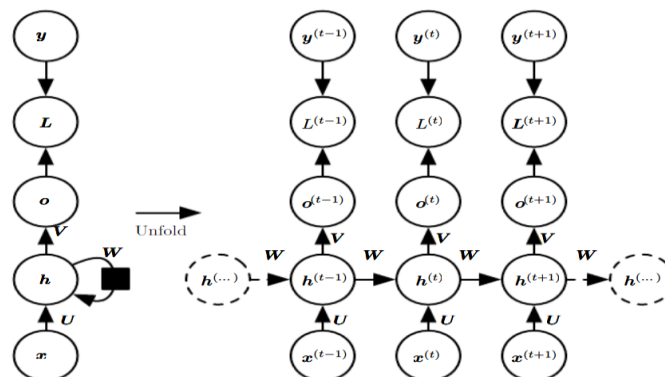


Fig. 7.1. Left: Folded computational graph of RNN. x – the input, o - the output values, y - the training target, h - hidden state, L - the loss function, h - the hidden state. Black filled rectangle shows time step. [Ref: “Deep learning” by Ian Goodfellow et. al.]

Exercise Questions:

Dataset contains Monthly and Daily prices of Natural gas, starting from January 1997 to current year. Prices are in nominal dollars. The task is to predict the price of natural gas using RNN model for the dataset

<https://datahub.io/core/natural-gas#resource-daily>

or

<https://www.kaggle.com/datasets/joebeachcapital/natural-gas-prices>

Given the price of last ten 10 days, corresponds to sequence_length, the RNN model must predict the price for the 11th day.

```
# importing the libraries
import numpy as np
import torch
import matplotlib.pyplot as plt
import pandas as pd
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from torch import nn
```

Prepare the data X[I to I+10] is the input X[I + 11] is the output:

```
df = pd.read_csv("./data/NaturalGasPrice/daily.csv")

# Preprocess the data - Drop NA values in the dataset
df = df.dropna()

y = df['Price'].values
x = np.arange(1, len(y), 1)

print(len(y))

# Normalize the input range between 0 and 1
minm = y.min()
maxm = y.max()
print(minm, maxm)
y = (y - minm) / (maxm - minm)
```

```
Sequence_Length = 10
```

```
X = []
Y = []
for i in range(0, 5900):
    list1 = []
    for j in range(i, i + Sequence_Length):
        list1.append(y[j])
    X.append(list1)
    Y.append(y[j + 1])
```

```
#Convert from list to array
```

```

X = np.array(X)
Y = np.array(Y)

#Split the data as the train and test set
x_train, x_test, y_train, y_test = train_test_split(X, Y,
test_size=0.10, random_state=42, shuffle=False, stratify=None)

class NGTimeSeries(Dataset):
    def __init__(self, x, y):
        self.x = torch.tensor(x, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)
        self.len = x.shape[0]

    def __getitem__(self, idx):
        return self.x[idx], self.y[idx]

    def __len__(self):
        return self.len

dataset = NGTimeSeries(x_train,y_train)
from torch.utils.data import DataLoader
train_loader = DataLoader(dataset,shuffle=True,batch_size=256)

```

#Create the RNN Model with input size as 1 and hidden_size as 5, with one hidden layer.
The input to RNN is of the shape (N, L, Hin) where N is the batch size, L is the sequence length, # and Hin the input_size

```

class RNNModel(nn.Module):
    def __init__(self):
        super(RNNModel,self).__init__()
        self.rnn =
nn.RNN(input_size=1,hidden_size=5,num_layers=1,batch_first=True)
        self.fc1 = nn.Linear(in_features=5,out_features=1)

    def forward(self,x):
        output,_status = self.rnn(x)
        output = output[:, -1, :]
        output = self.fc1(torch.relu(output))
        return output

model = RNNModel()

# optimizer , loss
criterion = torch.nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
epochs = 1500

```

```

# training loop
for i in range(epochs):
    for j, data in enumerate(train_loader):
        y_pred = model(data[:,0].view(-1, Sequence_Length,
1)).reshape(-1)
        loss = criterion(y_pred, data[:,1])
        loss.backward()
        optimizer.step()
    if i % 50 == 0:
        print(i, "th iteration : ", loss)

#test set actual vs predicted
test_set = NGTimeSeries(x_test,y_test)
test_pred = model(test_set[:,0].view(-1,10,1)).view(-1)
plt.plot(test_pred.detach().numpy(),label='predicted')
plt.plot(test_set[:,1].view(-1),label='original')
plt.legend()
plt.show()

#Undo normalization
y = y * (maxm - minm) + minm
y_pred = test_pred.detach().numpy() * (maxm - minm) + minm
plt.plot(y)
plt.plot(range(len(y)-len(y_pred), len(y)), y_pred)
plt.show()

```

2. The data provided at the link <https://download.pytorch.org/tutorial/data.zip> consists of names stored in the format {language: [names ...]} i.e. each language contains a list of names. Train a RNN on a few thousand surnames from 18 languages of origin, and predict which language a name is from based on the spelling.
3. Implement a next character predictor using RNN Model

Long- Short Term Memory Network

Objectives :

In this lab, student will be able to

1. Gain hands-on experience by implementing the architecture of LSTMs, emphasizing the recurrent connections and gates that allow the network to process sequential information with long-short memory.
2. Apply LSTMs to diverse tasks involving sequential data, such as time series prediction, natural language processing,

Long Short-Term Memory (LSTM) networks represent a specialized form of recurrent neural network (RNN) architecture tailored to tackle the vanishing gradient problem commonly observed in conventional RNNs. The vanishing gradient problem denotes the scenario where gradients diminish significantly as they are propagated backward through time during the training process, impeding the network's capacity to grasp long-term dependencies.

To mitigate this challenge, LSTMs incorporate a memory cell along with several gating mechanisms, enabling them to selectively retain or discard information across arbitrary time spans. The pivotal constituents of an LSTM unit encompass:

Cell State (C_t): Serving as the "memory" of the LSTM unit, the cell state traverses the entire sequence with minimal linear interactions. Visualized as a conveyor belt, the cell state facilitates the addition or removal of information, regulated by gate structures.

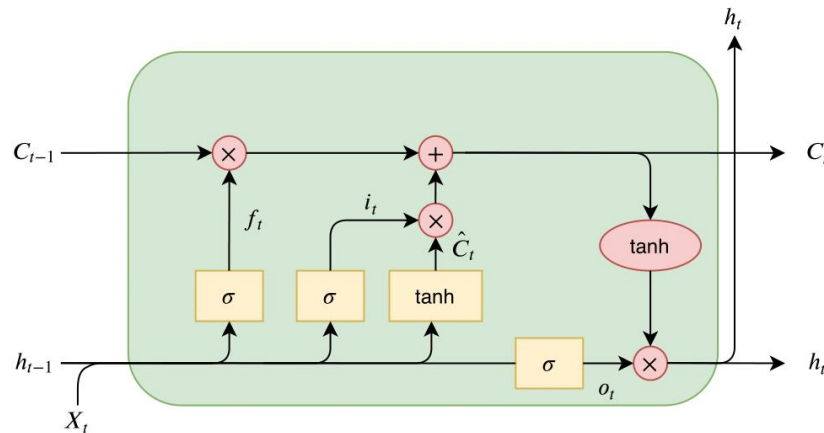
Forget Gate (f_t): Responsible for determining which information to discard or forget from the cell state, this gate evaluates the previous cell state and the current input, yielding values ranging from 0 to 1 for each element in the cell state. A value of 1 signifies retention, while 0 signifies elimination.

Input Gate (i_t): Tasked with determining the new information to be stored in the cell state, this gate processes the previous cell state and the current input through a sigmoid function. Additionally, it features another layer with a tanh activation function, generating a vector of potential new values to be incorporated into the state.

Output Gate (o_t): Dictating the information to be output based on the cell state, this gate processes the previous cell state and the current input through a sigmoid function. The cell state is also subjected to a tanh function. Ultimately, the output is determined by multiplying the tanh output with the sigmoid output.

These gates undergo training to regulate the information flow within the LSTM unit, enhancing its capability to learn long-term dependencies more effectively compared to conventional RNNs.

The following figure shows the architecture of a LSTM cell.



The computation of C_t , f_t , i_t , and o_t is summarized in the below equations.

$$\begin{aligned}
 i_t &= \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}) \\
 f_t &= \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf}) \\
 g_t &= \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}) \\
 o_t &= \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho}) \\
 c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\
 h_t &= o_t \odot \tanh(c_t)
 \end{aligned}$$

LSTMs find widespread application in domains such as speech recognition, natural language processing, time series prediction, and others, where accurate modeling of long-term dependencies is paramount.

Exercise Questions:

Dataset contains Monthly and Daily prices of Natural gas, starting from January 1997 to current year. Prices are in nominal dollars. The task is to predict the price of natural gas using LSTM model for the dataset

<https://datahub.io/core/natural-gas#resource-daily>

or

<https://www.kaggle.com/datasets/joebeachcapital/natural-gas-prices>

Given the price of last ten 10 days, corresponds to sequence_length, the LSTM model must predict the price for the 11th day.

```

# importing the libraries
import numpy as np
import torch
import matplotlib.pyplot as plt

```



```

import pandas as pd
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from torch import nn

```

Prepare the data $X[I \text{ to } I+10]$ is the input $X[I + 11]$ is the output:

```

df = pd.read_csv("./data/NaturalGasPrice/daily.csv")

# Preprocess the data - Drop NA values in the dataset
df = df.dropna()

y = df['Price'].values
x = np.arange(1, len(y), 1)

print(len(y))

# Normalize the input range between 0 and 1
minm = y.min()
maxm = y.max()
print(minm, maxm)
y = (y - minm) / (maxm - minm)

Sequence_Length = 10

X = []
Y = []
for i in range(0, 5900):
    list1 = []
    for j in range(i, i + Sequence_Length):
        list1.append(y[j])
    X.append(list1)
    Y.append(y[j + 1])

#Convert from list to array
X = np.array(X)
Y = np.array(Y)

#Split the data as the train and test set
x_train, x_test, y_train, y_test = train_test_split(X, Y,
test_size=0.10, random_state=42, shuffle=False, stratify=None)

class NGTimeSeries(Dataset):
    def __init__(self, x, y):
        self.x = torch.tensor(x, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)
        self.len = x.shape[0]

    def __getitem__(self, idx):
        return self.x[idx], self.y[idx]

    def __len__(self):

```

```

        return self.len

dataset = NGTimeSeries(x_train,y_train)
from torch.utils.data import DataLoader
train_loader = DataLoader(dataset,shuffle=True,batch_size=256)

#Create the LSTM Model with input size as 1 and hidden_size as 5, with one hidden layer.
# The input to LSTM is of the shape (N, L, Hin) where N is the batch size, L is the sequence
length, # and Hin the input_size

class LSTMModel(nn.Module):
    def __init__(self):
        super(RNNModel,self).__init__()
        self.lstm =
nn.LSTM(input_size=1,hidden_size=5,num_layers=1,batch_first=True)
        self.fc1 = nn.Linear(in_features=5,out_features=1)

    def forward(self,x):
        output,_status = self.lstm(x)
        output = output[:,-1,:]
        output = self.fc1(torch.relu(output))
        return output

model = LSTMModel()

# optimizer , loss
criterion = torch.nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
epochs = 1500

# training loop
for i in range(epochs):
    for j, data in enumerate(train_loader):
        y_pred = model(data[:,0].view(-1, Sequence_Length,
1)).reshape(-1)
        loss = criterion(y_pred, data[:,1])
        loss.backward()
        optimizer.step()
    if i % 50 == 0:
        print(i, "th iteration : ", loss)

#test set actual vs predicted
test_set = NGTimeSeries(x_test,y_test)
test_pred = model(test_set[:,0].view(-1,10,1)).view(-1)
plt.plot(test_pred.detach().numpy(),label='predicted')
plt.plot(test_set[:,1].view(-1),label='original')
plt.legend()
plt.show()

```

```
#Undo normalization
y = y * (maxm - minm) + minm
y_pred = test_pred.detach().numpy() * (maxm - minm) + minm
plt.plot(y)
plt.plot(range(len(y)-len(y_pred), len(y)), y_pred)
plt.show()
```

2. The data provided at the link <https://download.pytorch.org/tutorial/data.zip> consists of names stored in the format {language: [names ...]} i.e. each language contains a list of names. Train a LSTM on a few thousand surnames from 18 languages of origin, and predict which language a name is from based on the spelling.
3. Implement a next character predictor using LSTM Model

Auto-Encoders and Variational Encoder

Objectives :

In this lab, student will be able to

1. Implement Auto-encoders for dimensionality reduction and feature Learning.
2. Synthesize images, and data using Variational Auto-Encoders,

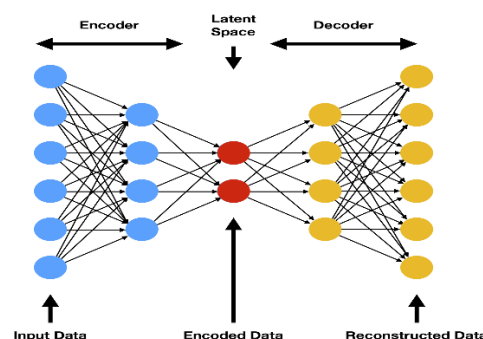
Autoencoders represent a category of artificial neural networks employed in unsupervised learning tasks. Their core concept revolves around acquiring a condensed version of the input data, termed the "encoding," followed by reconstructing the initial input data from this representation with high fidelity. Structurally, an autoencoder comprises an encoder and a decoder, typically designed to be symmetric

Autoencoders consists of the following components:

Encoder: The encoder takes the input data and maps it to a lower-dimensional latent space representation. This is achieved through a series of hidden layers that progressively reduce the dimensionality of the input data. The output of the encoder is the compressed representation of the input data, often referred to as the "encoding" or "latent representation."

Decoder: The decoder takes the compressed representation produced by the encoder and attempts to reconstruct the original input data from it. Like the encoder, the decoder consists of a series of hidden layers that progressively upscale the compressed representation back to the original dimensionality of the input data.

Loss Function: During training, the goal of the autoencoder is to minimize the difference between the input data and the reconstructed output. This is typically done by defining a loss function, such as mean squared error (MSE) or Mean Absolute Error (MAE), and optimizing it using techniques like gradient descent. The autoencoder learns to reconstruct the input data by adjusting the parameters of both the encoder and the decoder.



Autoencoders have several applications:

Dimensionality Reduction: By learning a compressed representation of the input data, autoencoders can be used for dimensionality reduction tasks, such as feature extraction or data visualization.

Data Denoising: Autoencoders can learn to reconstruct clean versions of noisy input data. This makes them useful for denoising tasks, where the goal is to remove noise from signals or images.

Anomaly Detection: Autoencoders can be trained on normal data and are expected to reconstruct it accurately. When presented with anomalous data during testing, they may fail to reconstruct it well, thus highlighting anomalies.

Feature Learning: Autoencoders can learn a set of meaningful features from the input data without explicit supervision. These learned features can then be used for downstream tasks like classification or clustering.

Variational Auto-Encoders:

Variational Autoencoders (VAEs) are a type of generative model that extends the concept of traditional autoencoders. They introduce probabilistic modeling into the encoding process, allowing for more structured and controllable generation of new data samples. Here's an overview of VAEs:

Probabilistic Encoding: Unlike traditional autoencoders, which directly map input data to a fixed-length encoding, VAEs map input data to a probability distribution in a latent space. This distribution is typically Gaussian with a mean and a variance.

Reparameterization Trick: To enable efficient training via backpropagation, VAEs employ a reparameterization trick. Instead of directly sampling from the learned distribution, the model samples from a standard Gaussian distribution and then applies a transformation based on the mean and variance learned by the encoder.

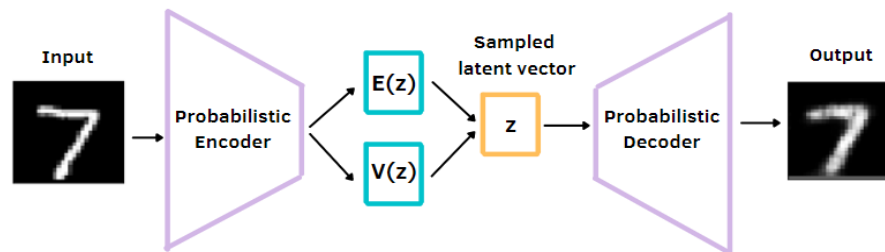
Generative Modeling: With the probabilistic encoding, VAEs can generate new data samples by sampling from the latent space distribution and passing the samples through the decoder. This allows for the generation of diverse and realistic-looking samples.

Objective Function: VAEs are trained by maximizing a variational lower bound on the log-likelihood of the data. This objective function consists of two terms: the reconstruction loss, which measures how well the model reconstructs the input data, and the KL divergence between the

learned latent space distribution and a prior distribution (typically a standard Gaussian). The KL divergence term encourages the learned distribution to be close to the prior distribution, acting as a regularization term.

Continuous Latent Space: VAEs learn a continuous and smooth latent space where each point corresponds to a generated sample. This allows for meaningful interpolations between points in the latent space, resulting in semantically meaningful transitions in the generated samples.

Applications: VAEs are widely used for generative modelling tasks such as image generation, text generation, and data imputation. They have also been applied in semi-supervised learning settings, where they can leverage both labelled and unlabelled data to improve model performance.



Exercise questions

1. Implement Auto-Encoder for latent representation of MNIST dataset.
2. Implement VAE for synthesizing digits using MNIST training data.

```

class VariationalAutoEncoder(nn.Module):
    def __init__(self):
        super().__init__()
        #self.w = torch.nn.Parameter(torch.rand([1]))
        #self.b = torch.nn.Parameter(torch.rand([1]))

        self.Encoder = nn.Sequential(nn.Linear(784,100,bias=True),
                                     nn.ReLU(),
                                     nn.Dropout(0.2),
                                     nn.Linear(100, 100, bias=True),
                                     nn.Sigmoid(),
                                     )
        self.mean = nn.Linear(100, 10, bias=True)
        self.std = nn.Linear(100, 10, bias=True)
        self.Decoder = nn.Sequential(nn.BatchNorm1d(10),
                                     nn.Linear(10, 100, bias=True),
                                     nn.ReLU(),
                                     nn.Linear(100, 100, bias=True),
                                     nn.ReLU(),
                                     nn.Linear(100,784,bias=True),
                                     nn.Sigmoid()
                                     )

    def forward(self, x):
        enc = self.Encoder(x)
        mean = self.mean(enc)
        std = self.std(enc)
        z = mean + std * torch.randn_like(std)
        y=self.Decoder(z)
        return mean, std, y.view(-1,28,28)
  
```

```
def loss_function(x, x_hat, mean, log_var):
    reproduction_loss = nn.functional.binary_cross_entropy(x_hat, x, reduction='sum')
    KLD = - 0.5 * torch.sum(1+ log_var - mean.pow(2) - log_var.exp())

    return reproduction_loss + KLD
```

#Write the dataloader for MNIST dataset

.....


```
def train_one_epoch(epoch_index):
    total_loss = 0.

    # Here, we use enumerate(training_loader) instead of
    # iter(training_loader) so that we can track the batch
    # index and do some intra-epoch reporting
    for i, data in enumerate(train_data_loader):
        # Every data instance is an input + label pair
        inputs, _ = data
        inputs = inputs.to(device)

        # Zero your gradients for every batch!
        optimizer.zero_grad()

        # Make predictions for this batch
        mean, std, outputs = model(inputs.view(batch_size,-1))

        # Compute the loss and its gradients
        loss = loss_fn(inputs, outputs, mean, std)
        loss.backward()

        # Adjust learning weights
        optimizer.step()

        # Gather data and report
        total_loss += loss.item()

    return total_loss/(len(train_data_loader) * batch_size)
```

```
def generate_digit():
    mean = torch.zeros((1,10))
    var = torch.ones((1, 10))
    z_sample = mean + var* torch.randn_like(var)
    x_decoded = model.Decoder(z_sample.to(device))
    digit = x_decoded.detach().cpu().reshape(28, 28) # reshape vector to 2d array
    plt.imshow(digit, cmap='gray')
    plt.axis('off')
    plt.show()

generate_digit()
generate_digit()
```


References:

1. Eli Stevens, Luca Antiga, and Thomas Viehmann, Deep Learning with PyTorch, Manning, 2020
2. Goodfellow, Ian, et al. Deep learning. Vol. 1. No. 2. Cambridge: MIT press, 2016.